

CHAPTER 2: PROJECT DESIGN

This chapter discusses the different design approaches considered for the development of the proposed system. This chapter presents the alternative designs and explains the structure and features of each design, including hardware and software components. It describes the design descriptions, schematics, illustrative layouts, applicable design standards, and design constraints. Through this discussion, the chapter helps compare the design options and explains the basis for selecting the most appropriate design for the project.

2.1 Discussion of Alternative Designs

The researchers initially developed three alternative system design approaches to address the storage and network infrastructure requirements of the INTRAK System. These approaches were formulated to explore different methods of managing internship documents and supporting the system's operational needs within an academic environment.

The first approach is a cloud-based architecture with NAS integration using a Raspberry Pi. The second approach is a fully cloud-based architecture without NAS integration, where both system services and document storage are hosted on cloud platforms to provide high availability and scalability. The third approach is a cloud-based architecture with dedicated NAS storage, in which all records are kept through a commercial NAS device that provides centralized storage, improved data security, reliable backups, and easier data management.

Each approach offers distinct advantages and limitations in terms of cost, data security, performance, scalability, and infrastructure requirements. By considering these three design alternatives, the researchers established a comprehensive basis for evaluating and selecting the most appropriate architecture for the successful implementation of the INTRAK System.

2.2 Design 1: Cloud-based with NAS Integration using a Raspberry Pi

2.2.2 Design Description

This design is hosted on cloud platforms, specifically Hostinger VPS (Virtual Private

Server), while documents are stored on an on-site Network-Attached Storage (NAS) system in Raspberry Pi. Users can access the application online, but sensitive records remain stored securely within the institution. NAS provides fast internal file access, redundancy, and compliance with school data rules. This approach balances remote access with security.

The INTRAK system includes several user roles that match the needs of the internship process. Each role has specific tasks and permissions to ensure that information is handled properly and responsibilities are clearly assigned. Students can submit requirements and track their internship status. Instructors and coordinators can review submissions, monitor progress, and give feedback. Host institutions can confirm attendance, evaluate performance, and communicate concerns. Administrators oversee the whole system, manage accounts, and ensure that all data remains accurate and up to date.

DEVELOPMENTAL TOOLS TO BE USE:

CATEGORY	TOOL
Frontend Framework	React 19 + Typescript
Styling Framework	Tailwind CSS
State Management	React Context API +Hooks
Routing	React Router v7
HTTP Client	Axios
Form Handling	React Hook Form + Zod
Build Tool	Vite
Backend Runtime	Node.js 22 LTS
Backend Framework	express.js
Backend Language	Typescript
ORM/ Database Tool	Prisma
Database	Postgres SQL

Authentication	jsonwebtoken
File Upload	Multer
Email Service	Nodemailer + SendGrid
PDF Generation	PDF kit
Excel Generation	Excel.js
Real-Time Communication	Socket.io
Validation Library	Zod
Testing Framework	Jest
Code Quality	ESlint + Prettier

Frontend Framework – React 19 + TypeScript

The INTRAK system adopts React 19 with TypeScript because it offers an efficient, component-driven architecture that fits well with modern modular web applications. React’s hooks-based design and concurrent features allow the system to remain fast, responsive, and easy to maintain as features grow. TypeScript strengthens the INTRAK architecture through static typing, fewer runtime errors, and better developer productivity.

Styling Framework – Tailwind CSS

Tailwind CSS is chosen for the INTRAK system by the researchers because its utility-first approach allows developers to build fully custom interfaces rapidly without managing large CSS files or fighting against predefined themes. It ensures consistent spacing, layouts, and design patterns across the system, while still giving the team full control over visual styling.

State Management – React Context API + Hooks

The proponents React Context API combined with Hooks is used in INTRAK because the application’s state needs are moderate and primarily revolve around global user

data and shared UI states. This native solution avoids bringing in heavier external libraries, making the codebase simpler and easier to maintain.

Routing – React Router v7

The system uses React Router v7 because it integrates naturally with React and provides modern routing capabilities such as nested routes, data loaders, and optimized navigation suited for SPAs. Its design aligns with React's component patterns, making route management intuitive and maintainable.

HTTP Client – Axios

INTRAK uses Axios for its clean syntax, automatic JSON transformation, built-in interceptors, and consistent request/response handling. It simplifies error management and integrates well with authentication flows.

Form Handling – React Hook Form + Zod

React Hook Form combined with Zod gives INTRAK fast, controlled form processing with minimal re-renders and full TypeScript-driven validation for the INTRAK system. This combination ensures accurate data entry and reduces bugs during user input.

Build Tool – Vite

The system uses Vite for its extremely fast hot-reload performance, efficient build pipeline, and native ES module support. Vite significantly speeds up developer iteration and produces optimized production bundles.

Backend Runtime – Node.js 22 LTS

Node.js is chosen because it enables JavaScript/TypeScript to run on both the client and server, simplifying development and reducing context switching. Its non-blocking event-driven model is ideal for real-time and I/O-heavy applications like INTRAK.

Backend Framework – express.js

INTRAK System uses express.js due to its simplicity, flexibility, and massive ecosystem of middleware. It allows developers to structure APIs with minimal boilerplate and maximum control.

Backend Language – TypeScript

The backend of the INTRAK system uses TypeScript to ensure strong typing, better tooling, and consistent language usage throughout the entire stack. This reduces logical errors, improves code readability, and speeds up development.

ORM / Database Tool – Prisma

Prisma is used because it provides a clean schema-driven workflow, auto-generated TypeScript types, and intuitive query APIs, greatly reducing the chance of database-related bugs.

Database – PostgreSQL

INTRAK uses PostgreSQL because it is a stable, feature-rich relational database with strong support for transactions, constraints, indexing, and advanced SQL capabilities. It is ideal for applications requiring reliable structured data.

Authentication – jsonwebtoken (JWT)

The proponents also uses jsonwebtoken because JWT authentication is lightweight, stateless, and easy to scale across distributed systems. It enables fast authorization without storing sessions on the server. Passport.js offer a wide range of strategies but are often more complex than needed.

File Upload – Multer

INTRAK uses Multer can provide simple, efficient, and well-documented file upload handling for Node.js. It works smoothly with Express middleware patterns.

Email Service – Nodemailer + SendGrid

The INTRAK system uses Nodemailer for local email generation and SendGrid for reliable external delivery. This combination ensures flexibility, scalability, and high deliverability rates.

PDF Generation – PDFKit

PDFKit is chosen for the INTRAK system because of its ability to generate PDFs programmatically with fine-grained layout control. It is ideal for dynamic documents such as reports or certificates.

Excel Generation – ExcelJS

INTRAK uses ExcelJS because it allows generation of fully formatted spreadsheets, complete with formulas, styling, and multiple sheets. This level of control is necessary for professional reporting.

Real-Time Communication – Socket.io

Socket.io is selected because it simplifies real-time communication through automatic reconnection, event abstraction, and fallback mechanisms to ensure consistent connectivity. This makes it ideal for notifications, live updates, and user interactions.

Validation Library – Zod

The INTRAK system uses Zod because it is TypeScript-first, allowing validation schemas to automatically generate type definitions. This reduces duplication, improves reliability, and fits naturally into a TypeScript-based frontend. Zod is lightweight, easy to read, and optimized for browser use.

Testing Framework – Jest

The system adopts Jest because it provides a complete testing ecosystem—runner, mocking, assertions, and snapshots—without requiring additional setup. It works reliably with React and TypeScript and has long-standing community support.

Code Quality – ESLint + Prettier

INTRAK uses ESLint + Prettier to enforce consistent code style and prevent common programming errors. ESLint offers mature React and TypeScript support, while Prettier ensures automatic formatting across all files.

2.2.2 Hardware/Network Design

Client Layer:

Student workstations, faculty computers, and mobile devices. The researchers used these functions as the primary access points for all users of the INTRAK system. Students and faculty access the platform through desktop computers, laptops, tablets, or smartphones to accomplish academic and administrative tasks. The researchers designed the system to accommodate a wide range of user devices to ensure accessibility, convenience, and consistent system interaction regardless of hardware differences.

Web browsers (Chrome, Firefox, Edge, Safari)

Web browsers will serve as the interface through which users interact with the INTRAK system. Browsers such as Chrome, Firefox, Edge, and Safari interpret the system's web content, process user inputs, and ensure stable rendering of system features. The proponents selected a browser-based interface to promote ease of use, platform independence, and compatibility across various operating systems.

Connect via Internet (HTTPS) to the Cloud Application

Communication between user devices and the INTRAK system occurs through the internet using the HTTPS protocol. This secure communication channel encrypts data exchanges, protecting sensitive information from unauthorized access. The development team implemented HTTPS to uphold data confidentiality, maintain integrity during transmission, and comply with established security and privacy requirements.

Application Layer (Cloud-Hosted):

Platform: Hostinger VPS (Virtual Private Server)

The researchers implemented the INTRAK system on Hostinger VPS, which hosts the frontend, backend API, and database on a single virtual private server. The frontend provides the React-based interface that delivers the system's pages, forms, and visual components to users. The backend Node.js application processes the system's logic, manages user requests, handles authentication, data operations, and communication with the database. The PostgreSQL database stores all system data, including records, submissions, and activity logs. The project team selected Hostinger VPS to ensure fast loading times, stable performance, dependable access to the system interface, and centralized management of all application components. This setup provides structured

data handling, automated maintenance capabilities, and reliable backup management while enhancing data security, maintaining consistency, and simplifying long-term data administration.

Architecture: Microservices / Decoupled Frontend–Backend

The project team adopted a decoupled architecture where the frontend and backend operate as separate but interconnected components. This approach allows each part to function independently while exchanging information through secure API calls. The chosen architecture improves system flexibility, supports easier updates, and allows future enhancements without disrupting existing functions.

Network-Attached Storage Layer:

Device: Raspberry Pi 4 Model B 4GB

The project team used a Raspberry Pi 4 Model B as the main hardware component for the system. Its available RAM options support stable performance and smooth execution of required tasks. The device provides sufficient processing capability for continuous system operation. This makes it suitable for lightweight server functions within the INTRAK system.

Operating System: Raspberry Pi OS:OpenMediaVault (Debian-based)

The Raspberry Pi runs Raspberry Pi OS with OpenMediaVault installed to manage storage and network functions. This combination provides a stable operating environment suited for handling system services. OpenMediaVault offers organized control of shared files, device settings, and storage operations. Together, they support efficient and reliable backend processes for the INTRAK system.

Connectivity

The device connects to the campus network through a Gigabit Ethernet link to ensure fast and stable communication. The Raspberry Pi NAS communicates with the Hostinger VPS through a Tailscale VPN tunnel, which provides secure, encrypted connectivity without requiring port forwarding or exposing the device to the public internet. This VPN-based approach maintains secure and consistent connectivity for backend operations while keeping the NAS within the campus network perimeter.

Network Infrastructure:

Cloud: Hostinger VPS Infrastructure

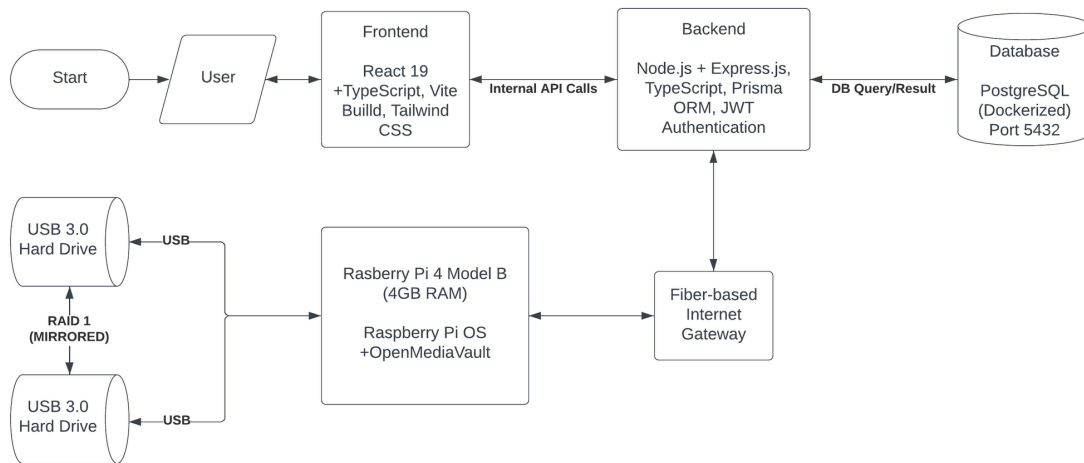
The system uses Hostinger VPS to host the frontend application, backend API services, and PostgreSQL database on a single virtual private server. This cloud platform provides stable performance, automatic scaling capabilities, and secure access for users. By hosting all system components on Hostinger VPS, the project team ensures that the application remains available even during high user activity. This setup also reduces hardware maintenance and improves long-term reliability through centralized management and monitoring.

Campus Infrastructure

The campus network uses a fiber-based internet gateway, an institutional firewall, and a structured LAN to support the system's operations. The fiber connection provides fast and stable access to the Hostinger VPS services, while the firewall manages security through NAT and port forwarding. The Raspberry Pi NAS is connected to the campus LAN and communicates with the Hostinger VPS through a Tailscale VPN tunnel, allowing it to communicate safely and efficiently with both internal users and the cloud-hosted application. Together, these components ensure secure, reliable, and continuous system connectivity.

2.2.3 Schematic Design

Cloud-based with NAS Integration using an Raspberry Pi



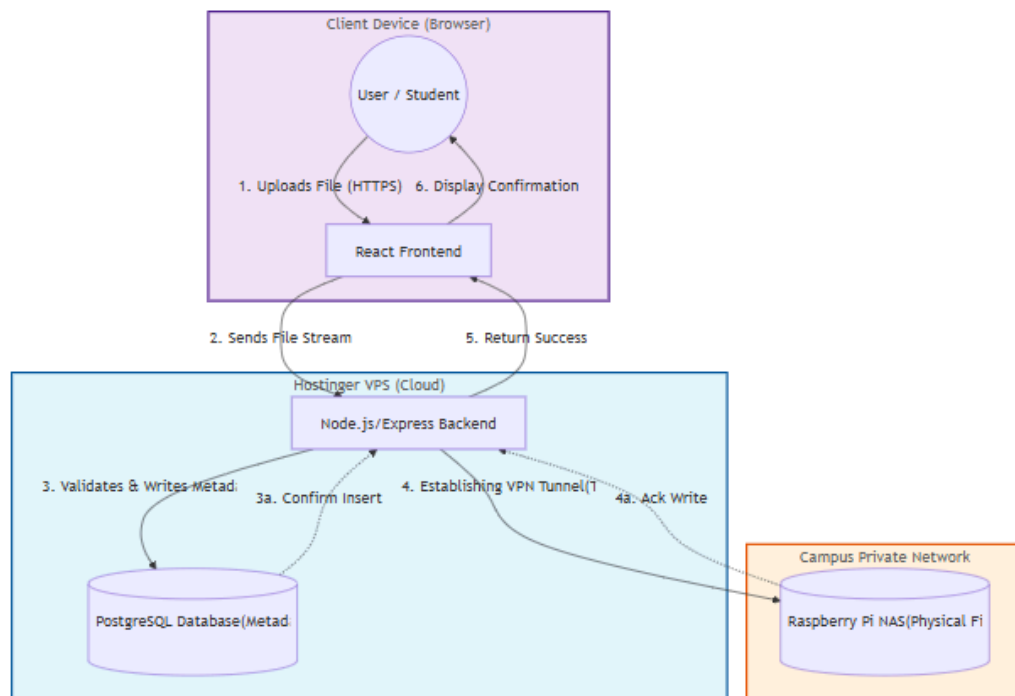
The flowchart provided illustrates the overall architecture and operational flow of the INTRAK System. This diagram was created to show how data moves across the different components of the system, beginning from the user interface and ending at the secure storage layer. It also visualizes how each part of the system works together to support reliable document processing for internship requirements.

The flow begins with the user, whose interactions are handled through the frontend interface built using React 19, TypeScript, Vite, and Tailwind CSS. All actions performed by the user—such as logging in, uploading documents, or viewing internship tasks—are transmitted from the frontend to the backend through internal API calls.

The researchers designed the backend using Node.js, Express.js, TypeScript, Prisma ORM, and JWT authentication. This layer processes system logic, validates user credentials, manages document transactions, and ensures secure communication within the system. The backend then interacts with the PostgreSQL database, which is deployed through Docker. This database stores user records, document metadata, and system logs, and exchanges query results with the backend as indicated in the flowchart.

For file storage, the researchers used a Raspberry Pi 4 Model B running Raspberry Pi OS with OpenMediaVault to serve as the local NAS-like storage server of the INTRAK System. This device is connected to the backend through the institution's fiber-based internet gateway, ensuring stable internal access to stored documents.

2.2.4 Illustrative Design



Design 1 of the INTRAK system integrates a cloud-hosted application with a Raspberry Pi NAS for secure document storage. Students access the system through a web interface over HTTPS, allowing them to submit pre-deployment, internship, and post-deployment documents. Uploaded files are validated by the backend, securely written to the NAS via a Tailscale VPN, and their metadata—including file path, filename, upload date, and user ID—is stored in a PostgreSQL database on the VPS. Upon successful submission, students receive confirmation, and coordinators may be notified via email.

Document retrieval and viewing follow a similarly secure process. The system verifies user permissions, retrieves metadata from the database, and streams the file from the NAS to the client over an encrypted VPN tunnel. All communications between client and server occur through HTTPS, ensuring data privacy and integrity. Metadata operations, including storing attendance logs, evaluations, and document information, are handled efficiently using PostgreSQL and Prisma ORM to maintain data consistency.

The system also incorporates backup and redundancy mechanisms to ensure data safety. Automated backups of the database are performed on the VPS, while NAS shares can be included in institutional backup routines. Separate schedules for VPS and NAS backups enhance disaster recovery capabilities

2.2.5 Design Standards

1. Clear Value Proposition and Purpose

INTRAK effectively communicates its purpose as an internship tracking and management system immediately upon entering the site. This clarity was intentionally applied by the researchers by prioritizing concise descriptions and simple visual elements on the homepage, ensuring that users understand what the system does before navigating further. This approach follows standard usability guidelines that emphasize early visibility of system purpose to reduce confusion, support intuitive navigation, and help new users quickly grasp the platform's function.

2. Simple and Consistent Navigation

The system demonstrates a straightforward and predictable navigation structure, which helps users access main features such as registration, document upload, attendance tracking, and reports with ease. The researchers applied common navigation conventions, including consistent menu placement and clear labeling across all pages. This uniformity aligns with web-design standards focused on enhancing usability and findability, enabling both students and coordinators to navigate the platform without unnecessary complexity.

3. Clean Interface and Organized Layout

The prototype presents a clean and structured interface, using adequate spacing, readable text, and logical grouping of related functions. The researchers applied basic visual hierarchy principles, ensuring that headings, form sections, and buttons follow consistent alignment and sizing. This approach improves readability and reduces cognitive load for users, which is supported by established UI design standards emphasizing simplicity and minimal visual clutter to enhance user comfort and task efficiency.

4. Focused Feature Presentation

Design 1 for INTRAK avoids unnecessary content and focuses solely on system components relevant to the internship workflow. The researchers intentionally designed the homepage and its subsequent pages to include only essential sections, preventing distractions and allowing users to move directly to their intended tasks. This focused presentation follows standard design practices that highlight the importance of clarity and content prioritization, ensuring that users can complete tasks more efficiently without navigating through irrelevant information.

The researchers conclude that the cloud-based NAS integration using a Raspberry Pi in the INTRAK system demonstrates strong alignment with widely accepted web design standards. The use of consistent branding, clear navigation, readable typography, and a well-structured layout supports a smooth and intuitive user experience. These strengths confirm that the design choices made—such as adopting a centralized navigation bar, applying uniform visual elements, and maintaining stable page layouts—were effective and appropriate for the needs of students, coordinators, and administrators. By integrating these practices, the system ensures clarity, reliability, and familiarity, contributing to efficient use and reducing the cognitive load on its users.

2.2.6 Design Constraints

1. Accessibility and WCAG Compliance

While this proposed design includes foundational accessibility features such as aria-labels on interactive elements, alt texts for images, and semantic HTML structure, some aspects may not yet fully comply with modern accessibility standards. Areas for improvement include verifying contrast ratios for all text and background combinations, enhancing keyboard navigation with visible focus states, and adding additional screen reader support through role attributes and aria-describedby elements. These enhancements are essential because accessibility standards, such as WCAG, ensure that all users—including those with visual or motor impairments—can use the system effectively, creating an inclusive and user-centered design.

2. Feedback and Notification System Enhancement

The system currently uses loading states, empty state messages, and confirmation modals to provide user feedback. However, the visibility of success and error messages after key actions could be improved. Implementing a more prominent toast notification system would provide immediate visual confirmation when users complete actions such

as uploading documents, submitting forms, or updating records. This improvement is meaningful because clear feedback reduces user uncertainty and aligns with design standards that emphasize responsive and informative user interactions.

3. Multi-Step Workflow Guidance

Some user workflows, particularly those involving multiple steps such as document submission or company application processes, could benefit from clearer progress indicators. Adding visual cues such as step counters, progress bars, or contextual "next step" prompts would help users understand where they are in a process and what actions remain. This enhancement follows design standards that promote intuitive navigation and reduce the cognitive load required to complete complex tasks.

4. Cross-Device Testing Documentation

Although the system demonstrates strong responsive design with mobile-specific layouts, collapsible sidebars, and adaptive grid structures, formal cross-device and cross-browser testing documentation would strengthen quality assurance. Establishing documented testing procedures for various screen sizes, browsers, and operating systems ensures consistent user experiences and helps identify edge cases that may require additional optimization.

5. Brand Kit Documentation

The INTRAK system maintains consistent branding through unified logo placement, a defined color palette, and standardized typography across all user portals. To support long-term maintenance and future development, creating formal brand kit documentation that specifies logo usage guidelines, color codes, typography rules, and spacing conventions would be beneficial. This documentation ensures that visual identity remains consistent as the system evolves and new features are added.

.

2.3 Design 2: Cloud-Based Angular–Spring Boot Web Platform for INTRAK

2.3.1 Design Description:

This design proposes a pure cloud-based web platform for the INTRAK system. It provides a clean and intuitive interface for students, instructors, host institutions, and

administrators, with smooth navigation and efficient management of user data. Internship documents and reports are stored securely in the cloud, supporting centralized, reliable, and redundant file management. The system also delivers real-time updates for submissions, approvals, and feedback. This design emphasizes scalability, reliability, and long-term usability, making it suitable for large academic organizations.

DEVELOPMENTAL TOOLS TO BE USE:

CATEGORY	TOOLS
Frontend Framework	Angular
Styling Framework	Bootstrap or Material UI
State Management	Redux Toolkit
Routing	Angular Routing
HTTP Client	Angular httpclient
Form Handling	Angular reactive forms
Build Tool	Maven / Cradle
Backend Runtime	Java Runtime Environment
Backend Framework	Java Springboot
Backend Language	Java
ORM/ Database Tool	Spring Data JPA / Hibernate
Database	MySQL + MongoDB
Authentication	Auth0 or passportjs
File Upload	Spring Boot MultipartFile
Email Service	JavaMailSender
PDF Generation	Itex / Apache pdfbox
Excel Generation	Apache POI
Real-Time Communication	websockets
Validation Library	Hibernate validator

Testing Framework	Junit and Mockito frontend Jasmine and Karma backend
Code Quality	Sonarquebe

Frontend Framework

The INTRAK system's user interface is built using Angular, which provides a structured framework for creating dynamic, single-page web applications. Angular facilitates smooth navigation between pages and allows the application to load content efficiently without reloading the entire page. This improves the user experience for students, instructors, host institutions, and administrators by providing a responsive and interactive interface.

Styling Framework

Bootstrap or Material UI is used to style the frontend, ensuring a clean, consistent, and visually appealing design. These frameworks provide prebuilt UI components such as buttons, tables, forms, and modals, which accelerate development while maintaining a professional look. The responsive design also ensures that the platform is accessible on various devices, including desktops, tablets, and mobile phones.

State Management and Routing

The system manages shared data, such as user sessions, internship statuses, and role-based information, through Redux Toolkit, which allows centralized state handling. For navigation between pages and modules, Angular Routing is implemented, enabling seamless transitions and proper URL management.

HTTP Client and Form Handling

All interactions between the frontend and backend are handled using Angular HttpClient, which allows secure and efficient API communication for submitting and retrieving data. Angular Reactive Forms are used to manage user input, validation, and submission in an organized manner. This approach ensures that data collected from students and administrators, such as internship applications, approvals, and evaluations, is properly validated and transmitted to the backend.

Build Tool

Maven or Gradle is used as the build tool for the project, automating the compilation, dependency management, and packaging of the application. These tools simplify project setup, streamline deployment, and ensure that all components are correctly integrated, which is critical for maintaining a large-scale cloud-based system like INTRAK.

Backend Runtime and Framework

The backend runs on the Java Runtime Environment and is built using Spring Boot, which provides a reliable and scalable platform for developing RESTful APIs. Spring Boot simplifies backend development by offering preconfigured templates, dependency injection, and modular architecture, which ensures the system can handle multiple simultaneous users and large volumes of internship data efficiently.

Database and ORM

The system uses MySQL for structured data, such as student profiles, internship records, and evaluation results, while MongoDB handles flexible document-related data. Database operations are managed through Spring Data JPA or Hibernate, which provide object-relational mapping (ORM) tools that simplify interaction with the database and ensure data consistency, security, and efficient querying.

Authentication and Security

User authentication and role-based access control are implemented using Auth0 or Passport.js, ensuring that students, instructors, and administrators can only access the features and data they are authorized to use. This enhances system security and protects sensitive internship information from unauthorized access.

File Management and Communication

Spring Boot MultipartFile is used to handle file uploads, while JavaMailSender manages email notifications for submissions, approvals, and feedback. The system can generate PDF documents using iText or Apache PDFBox, and Excel files using Apache POI, allowing structured reports and records to be distributed efficiently. WebSockets enable real-time communication, ensuring that updates such as submission confirmations and approval notifications are delivered instantly to users.

Validation, Testing, and Code Quality

Data validation is enforced through Hibernate Validator, ensuring that input from users meets predefined rules before being processed. Testing is conducted using JUnit and Mockito for backend logic and Jasmine and Karma for frontend components, which guarantees that both the server-side and client-side functionalities operate correctly. SonarQube is used to maintain code quality by analyzing code for potential bugs, vulnerabilities, and code smells, promoting maintainable and reliable software.

2.3.2 Hardware/Network Design

The hardware and network architecture of the this chosen architecture for the INTRAK System is organized into three primary layers to ensure scalability, reliability, and role-based access:

Client Layer:

End-users access the system via web browsers on desktops, laptops, or mobile devices. The frontend is built with Angular and styled using Bootstrap or Material UI, providing responsive and consistent user interfaces for students, instructors, coordinators, administrators, and host institution supervisors. The Angular application uses Angular Router for page navigation and Redux Toolkit for managing shared application state, including user sessions and internship status.

Application Layer (Backend):

The backend runs on Java Spring Boot, deployed either on a cloud server or an institutional server. It handles business logic, user authentication, role-based access control, and communication with the database and storage systems.

Auth0 or Passport.js is integrated for secure authentication, ensuring that each role only accesses authorized features. WebSockets or Server-Sent Events (SSE) are used to provide real-time notifications, such as document submission confirmations, approval alerts, and evaluation feedback.

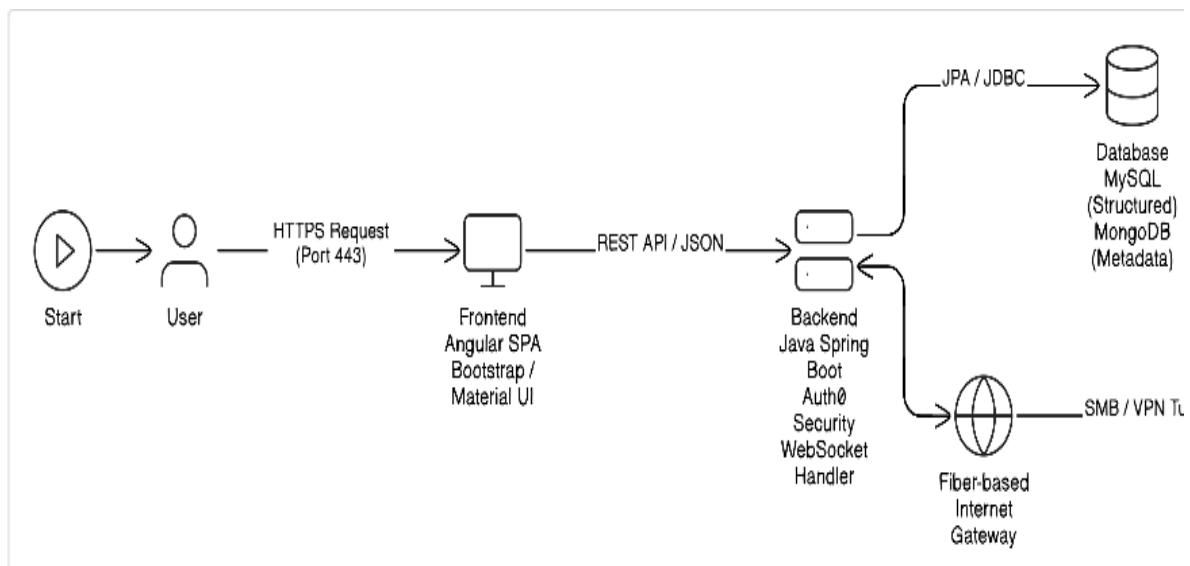
Data/Storage Layer:

Structured data, including student profiles, internship records, evaluations, and approvals, is stored in MySQL, while document metadata can be stored in MongoDB for flexibility.

Internship documents and reports are kept in a commercial NAS device (e.g., Synology, QNAP) or optionally cloud-based storage, ensuring centralized, reliable, and redundant storage. The storage layer also handles backup and redundancy, preventing data loss and supporting continuous system availability.

2.3.3 Schematic Design

Cloud-Based Angular–Spring Boot Web Platform for INTRAK

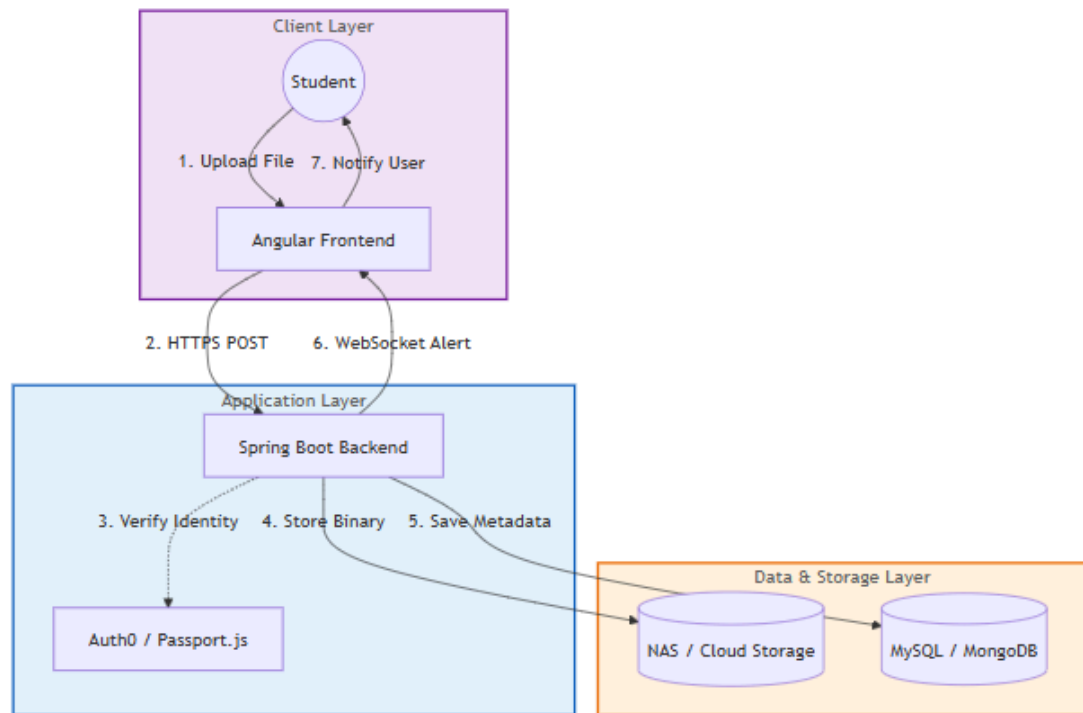


The INTRAK system is organized into three layers for scalability, reliability, and secure role-based access. The client layer allows students, instructors, coordinators, administrators, and host institution supervisors to access the system through web browsers, with a responsive interface and consistent navigation. The application layer handles business logic, user authentication, and role-based access, while providing real-time notifications for submissions, approvals, and evaluations. The data and storage layer manages structured data in MySQL, flexible document metadata in MongoDB, and internship files in a commercial NAS or cloud storage, ensuring centralized, reliable, and redundant storage with backup and continuous availability.

2.3.4 Illustrative Design

The illustrative design demonstrates the data flows, network communication, and storage operations in the enterprise INTRAK system:

1. Document Upload Process



Design 2 illustrates how the INTRAK system handles internship documents from upload to retrieval. Students submit files through the Angular frontend, which are stored securely in NAS or cloud storage, while metadata like filenames, timestamps, and submission status are recorded in MySQL or MongoDB. Real-time notifications via

WebSockets or Server-Sent Events (SSE) inform users about successful uploads and updates.

The system enforces role-based access control so only authorized users can access documents, ensuring security and accountability. Following a three-tier architecture, it separates the client, application, and data layers, supporting modularity, maintainability, and usability. Standards for security, data integrity, and web accessibility are applied throughout, making the system reliable and user-friendly for students, instructors, and coordinators.

2.3.6 Design Constraints

Technical Complexity:

The system requires expertise in multiple technologies, including Angular, Spring Boot, MySQL, MongoDB, and WebSockets/SSE. Integrating these technologies into a single enterprise system increases development complexity, requires skilled developers, and may extend development time compared to simpler architectures. Debugging and maintaining such a complex system can also be challenging, especially if team members are not equally proficient in all required technologies.

Hardware Limitations:

The cloud-based design is limited by the hardware resources provided by the VPS, including processing power, memory, and storage capacity. Since no local NAS device is used, the system relies entirely on cloud infrastructure, limiting direct hardware control and requiring subscription upgrades for resource expansion. System performance and availability are dependent on internet connectivity and shared cloud resources, which may affect reliability during high-traffic periods.

Network Dependency:

The system relies heavily on stable internet and network connectivity, particularly for cloud-hosted components and real-time notifications. Network downtime, latency, or slow connections can disrupt student submissions, coordinator reviews, and real-time updates, temporarily affecting system usability. Limited bandwidth in certain locations may also impact accessibility for remote users.

Cost and Licensing:

The design relies entirely on cloud-based storage and hosting, reducing the need for physical hardware but introducing recurring costs for server subscriptions, storage space, and bandwidth. Licensing costs may apply for software frameworks, database systems, or third-party services. Ongoing operational expenses may escalate as the user base grows, making cost management crucial for long-term sustainability.

Scalability Constraints:

Although the design is suitable for large academic institutions, handling very high numbers of concurrent users may require server upgrades, load balancing, or horizontal scaling. Without these enhancements, system performance may degrade under heavy usage, particularly during peak submission periods. Scalability limitations may also affect the responsiveness of real-time features such as notifications and collaborative tools.

Data Privacy and Compliance:

Handling student internship records requires compliance with institutional policies and data privacy regulations. These constraints may limit integration with certain third-party tools or cloud services and require strict monitoring and enforcement of access controls to protect sensitive information. Failure to comply with regulations could lead to legal or reputational consequences.

System Reliability and Maintenance:

As a cloud-based solution, the system's reliability is subject to the availability and stability of the hosting provider. Scheduled maintenance or unexpected outages on the server side could temporarily impact system operations. Additionally, updates to the frontend, backend, or database require careful planning to prevent downtime or data inconsistencies.

User Experience Constraints:

The complexity of the system may pose usability challenges for some users, particularly students or coordinators who are less familiar with online platforms. Training, clear

documentation, and intuitive design are necessary to mitigate potential confusion and ensure efficient use of the system.

Security Limitations:

While cloud infrastructure provides baseline security, potential vulnerabilities exist in web applications, such as cross-site scripting (XSS), SQL injection, or unauthorized access. Continuous monitoring, regular security audits, and implementation of best practices are required to protect sensitive internship data from cyber threats.

2.4 Design 3: Cloud-based with dedicated NAS Storage using Python Full Stack

2.4.1 Design Description:

Design 3 proposes a cloud-based INTRAK system with dedicated NAS storage using a Python full-stack approach. The system provides a flexible and customizable interface, with structured data stored in MySQL and document-related information in MongoDB. User authentication and role management ensure secure access for all users. Internship files are stored on a dedicated NAS, with optional cloud backup for redundancy, and real-time updates are delivered through WebSockets. This design emphasizes flexibility, rapid development, and maintainability, making it ideal for academic and research-focused implementations.

CATEGORY	APPLIED TOOL
Frontend Framework	Vue.js or Next.js
Styling Framework	Bootstrap or Material UI
State Management	Pinia (for Vue) or Redux/Context (for Next.js) (<i>Implicit</i>)
Routing	Vue Router or Next.js App Router (<i>Implicit</i>)

HTTP Client	Fetch API or SWR
Form Handling	Formik with Yup or Joi
Build Tool	Vite (for Vue) or Webpack (Next.js)
Backend Runtime	Python 3
Backend Framework	Django or Flask
Backend Language	Python
ORM/ Database Tool	Django ORM (if Django) or SQLAlchemy (if Flask)
Database	MySQL (Primary). MongoDB (Metadata)
Authentication	Auth0 or Passport.js
File Upload	Python shutil/SMB Protocol (to connect to NAS)
Email Service	Django Mail or Flask-Mail (Standard)
PDF Generation	ReportLab or WeasyPrint (Standard Python)
Excel Generation	Pandas or OpenPyXL (Standard Python)
Real-Time Communication	WebSockets (Django Channels or Flask-SocketIO)
Validation Library	Yup/Joi (Frontend) / Marshmallow (Backend)
Testing Framework	PyTest (Backend) / Jest (Frontend)
Code Quality	Pylint, Black (Python) / ESLint (Frontend)

Frontend Framework

The frontend of Design 2 is developed using Vue.js or Next.js, providing a dynamic and responsive interface for students, instructors, host institutions, and administrators. Navigation and routing are managed using Vue Router or Next.js App Router, allowing smooth transitions between pages. Styling frameworks like Bootstrap or Material UI ensure a professional and consistent layout. State management tools such as Pinia (for

Vue) or Redux/Context (for Next.js) maintain shared data like user sessions and internship status across components.

Form Handling and Build Tool: User input is managed with Formik combined with Yup or Joi, ensuring data entered in forms like applications and evaluations is validated before submission. This reduces errors and improves the integrity of data sent to the backend. Front-end code is built and optimized using Vite for Vue or Webpack for Next.js, streamlining deployment and improving performance.

Backend Runtime and Framework

The backend is implemented using Python 3 with Django or Flask frameworks, which handle server-side logic and user requests. Database interaction is facilitated through Django ORM or SQLAlchemy, allowing structured data in MySQL and flexible metadata in MongoDB to be efficiently managed. This setup ensures reliability in storing and retrieving student profiles, internship records, and evaluation data.

Security, File Management, and Real-Time Communication

Authentication and role-based access are enforced using Auth0 or Passport.js, providing secure login and authorization. File uploads are handled through Python shutil or SMB protocol to connect to cloud storage, while email notifications use Django Mail or Flask-Mail to inform users of updates. Real-time updates are provided with WebSockets (via Django Channels or Flask-SocketIO), keeping users informed about submissions, approvals, and feedback.

Reporting, Testing, and Code Quality

The system generates PDF reports using ReportLab or WeasyPrint and Excel files using Pandas or OpenPyXL for documentation and evaluation purposes. Testing is carried out using pytest for backend logic and Jest for frontend components, ensuring reliability and correct functionality. Code quality is maintained using Pylint and Black for Python and ESLint for frontend code, enforcing consistency, readability, and maintainability.

2.4.2 Hardware/Network Design

Client Layer:

Users interact with the system via web browsers on desktops, laptops, or mobile devices. The frontend is developed using Vue.js 3 or Next.js and styled with Material UI

or Bootstrap for a responsive and consistent interface. Form handling and validation are implemented using Formik with Yup or Joi to ensure accurate submissions. All data communication with the backend is performed using Fetch API or SWR, with the NAS acting as the central repository for document storage and retrieval, ensuring users always access up-to-date files.

Application Layer (Backend):

The backend is implemented using Python frameworks such as Django or Flask and hosted on a cloud or institutional server. It handles business logic, authentication, role-based access control, and workflow management. Real-time updates are delivered through WebSockets, while the NAS provides persistent and secure storage for all submitted documents. API endpoints manage communication with both the frontend and the NAS, ensuring smooth synchronization of structured and unstructured data.

Data/Storage Layer:

Structured data, including student profiles, internship records, and evaluations, are stored in MySQL, while flexible document metadata is stored in MongoDB. The Synology DS223 NAS stores all internship documents, providing centralized, redundant, and secure storage. Optional cloud backup can complement the NAS for additional redundancy, ensuring data is never lost and is easily recoverable.

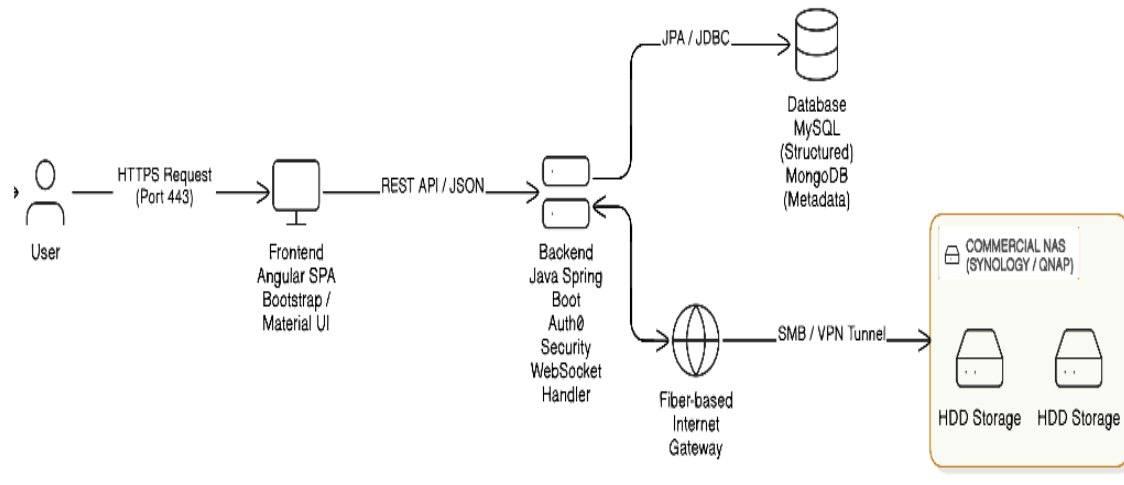
Integration/Service Layer:

Third-party services and system integrations, such as authentication (Auth0/Passport.js), email notifications, and optional cloud storage services, interact seamlessly with the NAS. The NAS enables modular integration by serving as a central hub for file storage while ensuring external services can operate without affecting core backend processes.

Security Layer (Cross-Cutting):

Security is enforced across all layers. HTTPS encrypts client-server communication, file transfers to and from the NAS are secured, and role-based access control ensures that only authorized users can access certain documents and system features.

2.4.3 Schematic Design

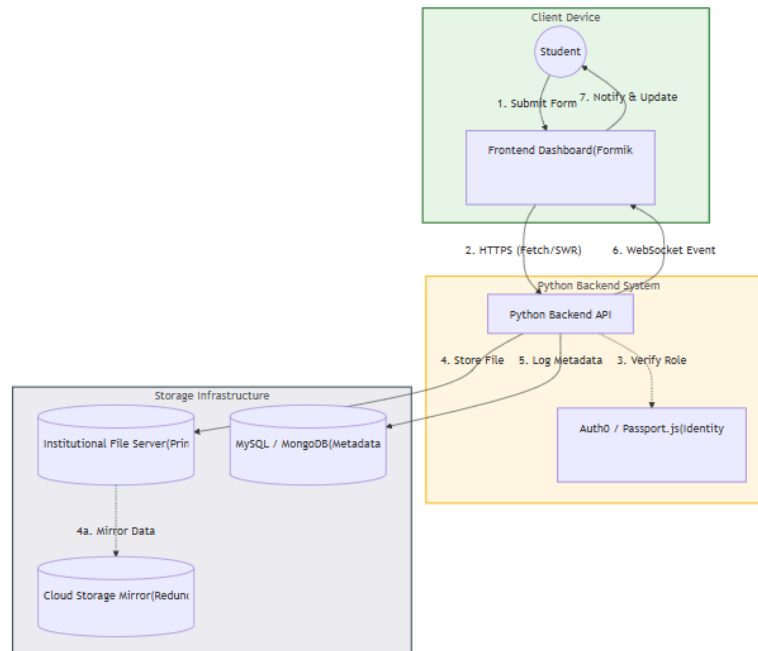


The schematic design of the INTRAK system illustrates a multi-layered architecture that integrates frontend, backend, data storage, and service components with a Synology DS223 NAS at its core. Users interact with the system through web browsers on various devices, accessing a responsive interface that handles forms, navigation, and real-time updates efficiently. The backend, implemented with Python frameworks, manages business logic, authentication, and workflow, while API endpoints synchronize data between the frontend and the NAS. Structured and flexible data are stored in MySQL and MongoDB, with all internship documents securely maintained on the NAS, complemented by optional cloud backup for redundancy. Third-party services such as authentication, email notifications, and optional cloud storage integrate seamlessly with the system, and security is enforced across all layers through HTTPS, role-based access control, and NAS-managed file permissions, ensuring reliable, scalable, and secure operations.

2.4.4 Illustrative Design

The illustrative design of the Python-Based Full Stack INTRAK System describes all major workflows, metadata operations, network communication, and redundancy mechanisms:

1. Document Upload Process



2.4.6 Design Constraints

Hardware and Storage Constraints: The system relies on a Synology DS223 2-bay DiskStation® NAS, which limits storage capacity to the physical drives installed. This constrains the number and size of internship documents that can be stored locally. NAS performance can also be affected by network bandwidth, especially when multiple users access large files simultaneously. The system's responsiveness and efficiency depend on proper NAS configuration, network speed, and concurrent access handling.

Cost Constraints: Implementing a commercial NAS involves initial purchase costs, including the NAS device, hard drives, and optional expansions. Ongoing costs include electricity consumption, maintenance, software updates, and potential licensing for advanced features. Institutions must plan for future storage expansions as internship records grow over time, which may incur additional costs for drives or upgraded NAS models.

Software and Compatibility Constraints:

The system relies on Python 3 with Django or Flask for the backend and Vue.js or Next.js for the frontend. These frameworks require compatible runtime environments and proper management of dependencies. Database operations in MySQL and MongoDB require careful configuration to maintain data consistency, support queries efficiently, and ensure integration with the NAS. Any software updates must be compatible with both the backend, frontend, and NAS system to prevent downtime.

Security and Compliance Constraints

The system must enforce role-based access control (RBAC), secure file transfers, and encryption via HTTPS to protect sensitive internship data. Files stored on the NAS must be properly permissioned to prevent unauthorized access. Backup and redundancy mechanisms are critical; failure to maintain them could result in data loss or violations of institutional data protection policies. Security monitoring and regular updates are also required to mitigate potential vulnerabilities.

Operational Constraints

The INTRAK system must deliver reliable real-time communication, workflow notifications, and document handling even under heavy load. NAS maintenance, including drive health checks and firmware updates, is necessary to prevent downtime. Network stability is also a critical factor, as interruptions can affect file access and workflow continuity. The system must balance performance, accessibility, and security to support institutional usage and large numbers of simultaneous users.

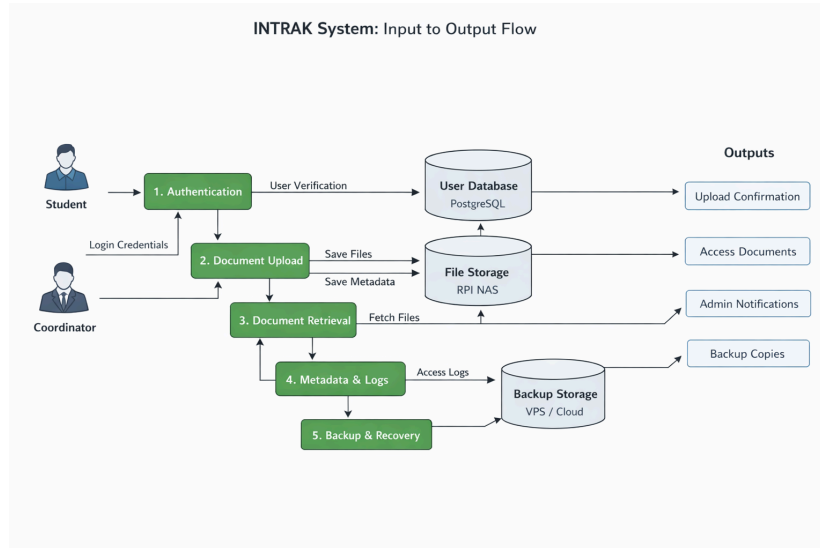
2.5 Software Design

This section describes the design of the system's software component in accordance with the defined project requirements. It emphasizes how functional and non-functional requirements are translated into a structured, interactive software solution that supports user operations and system processes. The software design is composed of the system algorithm, data flow diagram, system flowchart, and software user interface design which collectively illustrate the logical processing, data movement, and overall operational flow of the system.

2.5.1 Software Algorithm

This section explains the logical sequence of operations performed by the system during its execution.

2.5.2 DATA FLOW DIAGRAM



User Input

The process begins when a student or coordinator provides input to the system. This input can be login credentials for authentication or documents to be uploaded. The system uses these inputs to determine the user's actions and access level.

Authentication

Login credentials are verified against the PostgreSQL database to ensure that only authorized users can access the system. If the credentials are correct, the user is granted access to upload or retrieve documents. Unauthorized users are denied access, keeping the system secure.

Document Upload

Once authenticated, students can upload their documents along with the selected document type. The system validates the uploaded files for type, size, and permissions before storing them in the NAS. At the same time, metadata such as the file name, file

path, upload date, and user ID is saved in the PostgreSQL database, keeping track of both the physical file and its information.

Document Retrieval

When a user requests access to a document, the system first retrieves the file metadata from the database and then fetches the corresponding file from the NAS. The system checks permissions to ensure that users can only access documents they are allowed to. The document is then securely delivered to the user via HTTPS for viewing or download.

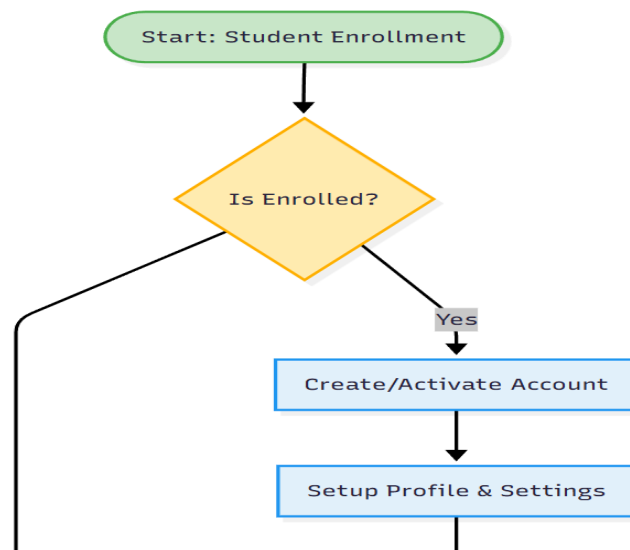
Logging and Monitoring

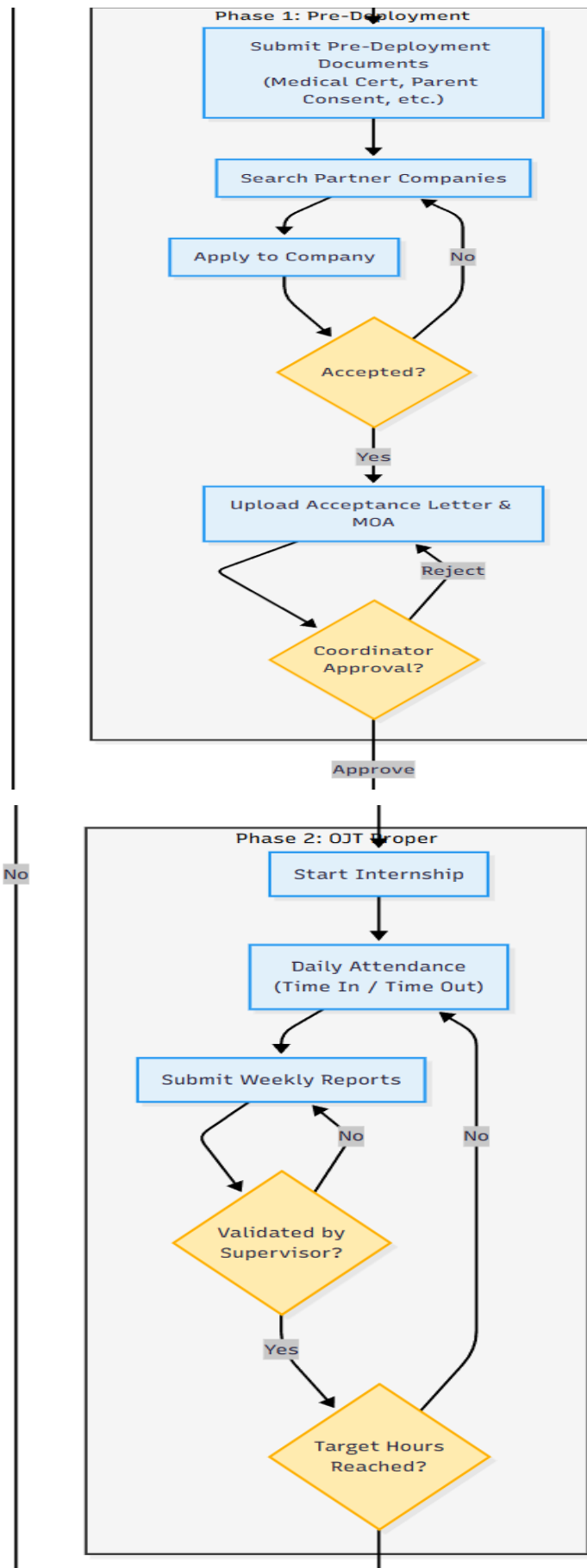
All user actions, including uploads, downloads, and document access, are recorded in the system. This logging creates an audit trail that allows administrators to monitor activity, maintain accountability, and ensure proper use of the system.

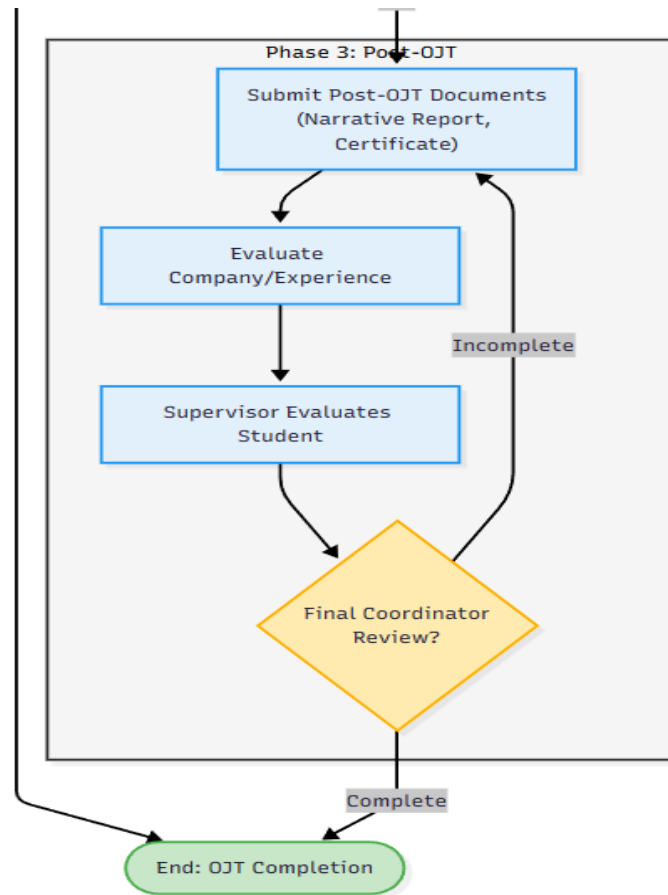
Backup

The system performs regular backups of both files and metadata to secondary storage, such as a VPS or cloud. This ensures data redundancy and protects the system from data loss due to hardware failure or accidental deletion.

2.5.3 SYSTEM FLOWCHART







The system begins checking if the student is enrolled and eligible for OJT deployment. If the student is not eligible, the system stops further processing and completes all necessary requirements. Eligible students proceed to the pre-deployment phase.

Phase 1 – Pre-Deployment Document Submission

Students submit required pre-deployment documents, and the system processes the Memorandum of Agreement (MOA). MoA will now be signed and both parties must agree with each of the conditions given. Once signed, they are now to be endorsed by the coordinator to the company. Conditional checks ensure that all documents are complete and valid. If any document is missing or incorrect, it will be immediately observed by the instructor otherwise the student will be now deployed to the company.

Phase 2 – Internship / Deployment

During this phase, the student's internship activities are recorded. The system tracks weekly reports and time records. The system logs all submissions and maintains a

record of student progress. Students must complete the required internship hours. Once completed and finished, the student will now proceed to phase 3.

Phase 3 – Post-Deployment Evaluation

After the internship, students submit post-deployment documents, including evaluation reports. The coordinator reviews these submissions, and the system consolidates the data to generate a final evaluation. Conditional decisions ensure that all documents are present; if not, the system requests completion before final assessment.

2.5.4 System User Interface Design

The user interface (UI) design of the INTRAK system provides a visual and functional platform for users to interact with the software efficiently. It is designed to be intuitive, allowing students, instructors, coordinators, host institutions, and administrators to navigate the system with ease. The UI presents clear menus, dashboards, forms, and notifications, guiding users step by step through document submission, progress tracking, and evaluation processes.

The system is primarily accessed through desktop and laptop platforms, enabling users to interact via standard web browsers. The interface is responsive and organized to support common display resolutions, ensuring usability across different devices, while future adaptation to mobile platforms is possible if needed.

Design 1 is built using a cloud-based architecture integrated with a Raspberry Pi NAS. The interface leverages general web development frameworks to manage user interactions, document uploads, retrievals, and real-time notifications, without requiring users to handle the underlying system operations directly.

The design also follows relevant software design standards, emphasizing usability, accessibility, and secure handling of data. This includes clear labeling of buttons and forms, structured workflows for tasks, and role-based access control to ensure that users see only the information and options relevant to their responsibilities.

STUDENT DASHBOARD

INTRAK
Student Portal

Overview

Documents

Companies

Find Company

Attendance

Evaluations

Reports

Notifications

Brandon Romlee S. Ilao

Logout

Welcome back, Brandon Romlee S. Ilao!

ID: 21-48-0288

Track your Internship progress and manage your requirements

Documents
0/0
Completed

Hours Completed
0
of 240 hours

Avg Rating
N/A
No evaluations

Progress
0.0%
On track

Internship Progress

0/240 hours

Document Status

No documents uploaded yet

Upload Document

Download Templates

Recent Attendance

No attendance records found

View Full Calendar

Internship Details

Company
Not assigned

Supervisor
N/A

Programs

INTRAK
Student Portal

Overview

Documents

Companies

Find Company

Attendance

Evaluations

Reports

Notifications

Brandon Romlee S. Ilao

Logout

Weekly Report (Form FM-AA-INT-17)

Fill out your tasks accomplished and knowledge, skills, values learned for each week

Save Export

Week 1

Date Range
e.g., June 17 - June 20, 2025

Tasks Accomplished
Enter tasks accomplished for this week...

Knowledge, Skills, Values Learned
Enter knowledge, skills, and values learned...

Add Week 2

Weekly Report Guidelines

Fill out each week with your tasks accomplished and what you learned

Be specific and detailed in your descriptions

Save your progress regularly

Export the report when all weeks are completed

INTRAK
Student Portal

Overview

Documents

Companies

Find Company

Attendance

Evaluations

Reports

Notifications

Brandon Romlee S. Ilao

Logout

OJT Requirements Checklist

Track your progress, download templates, and upload documents

COMPLETED 0 PENDING 0 TOTAL 0

Search requirements...

Pre-Deployment Requirements 8 Items

Record File

Missing

Upload

Application for Internship (Form FM-AA-INT-01)

Missing

Template

Upload

Medical Certificate and Psychological Test

Missing

Upload

Certification of Units Earned (Form FM-AA-INT-02)

Missing

Upload

Internship Resume (Form FM-AA-INT-09)

Missing

Upload

Consent Form (Form FM-AA-INT-03)

Missing

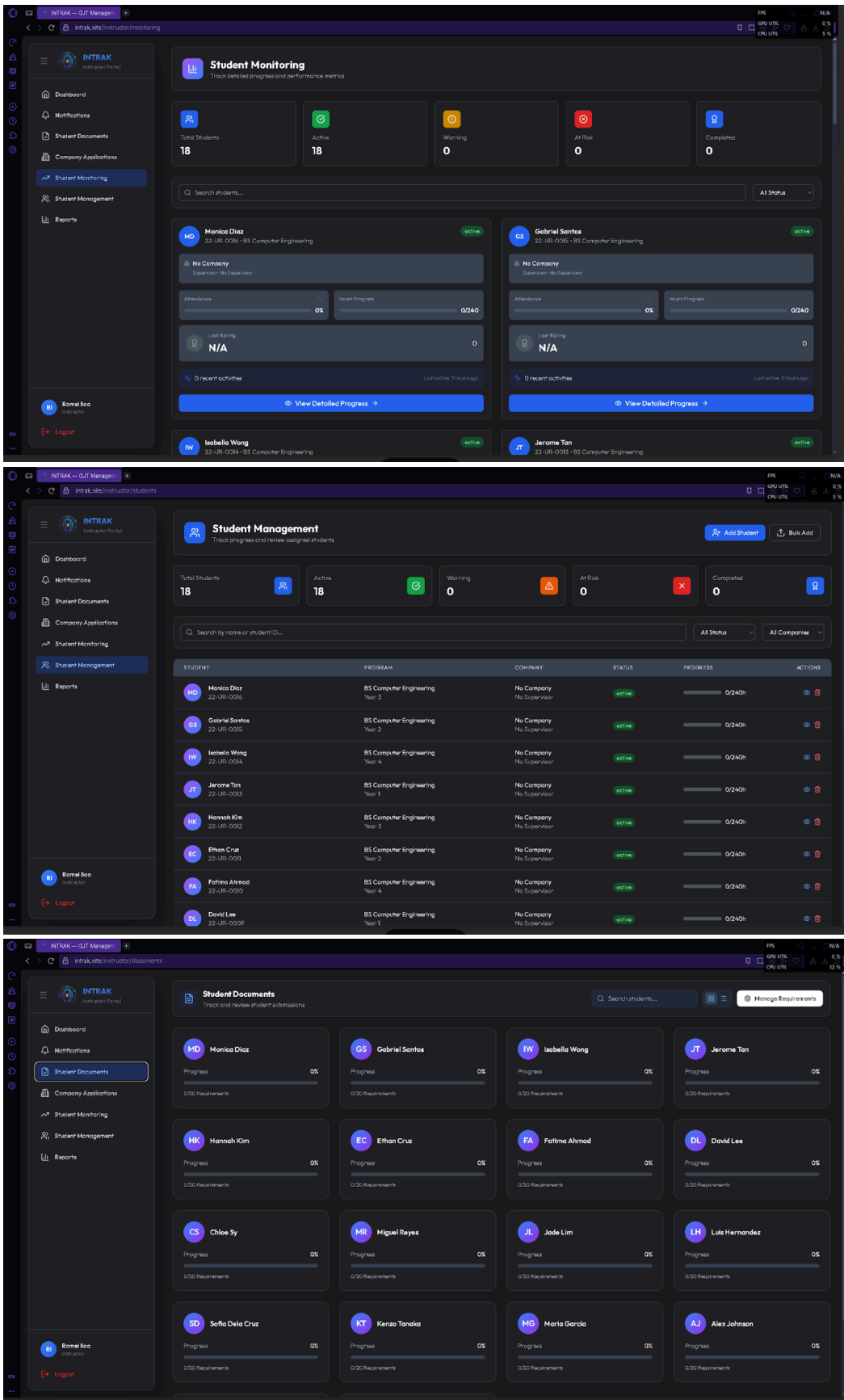
Upload

Endorsement Letter (Form FM-AA-INT-05)

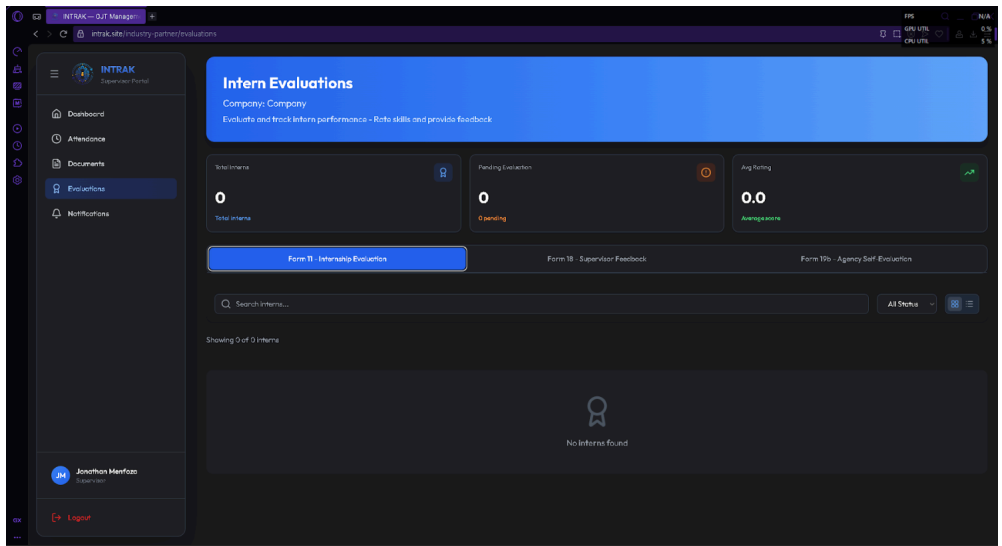
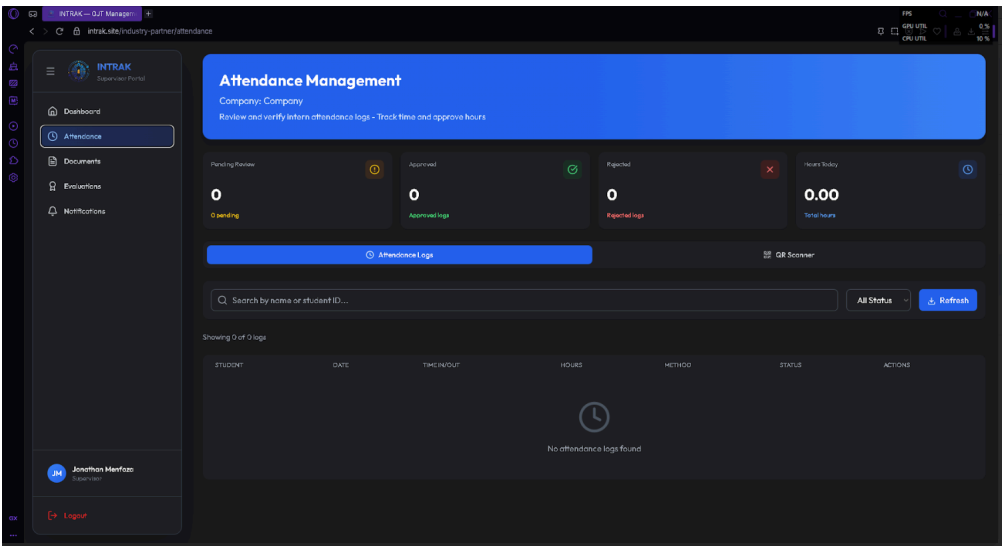
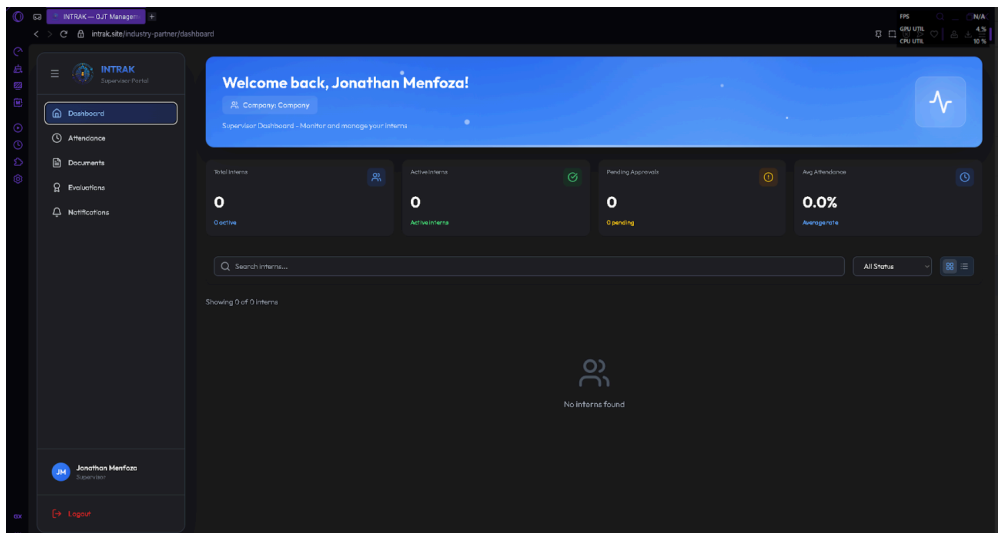
Missing

Upload

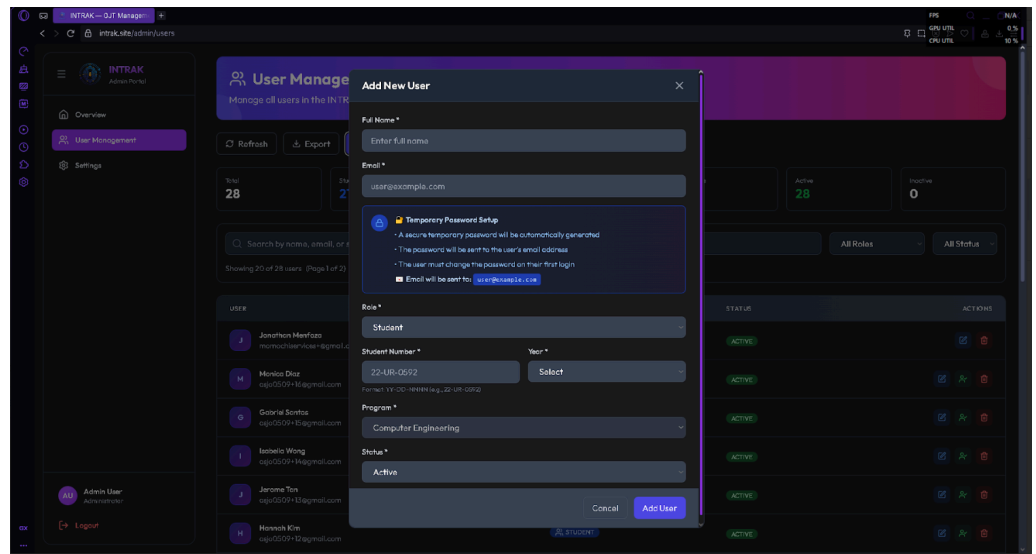
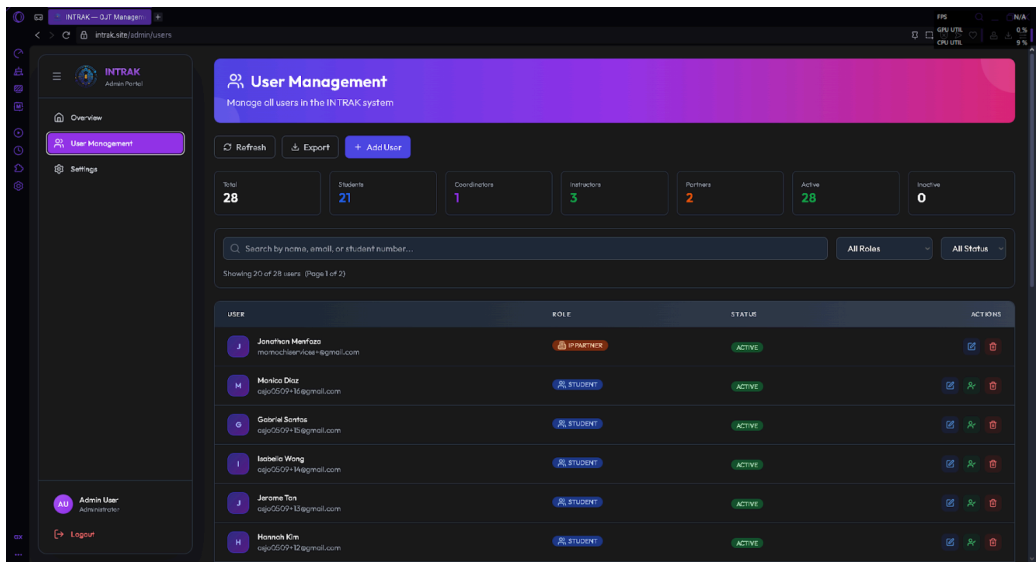
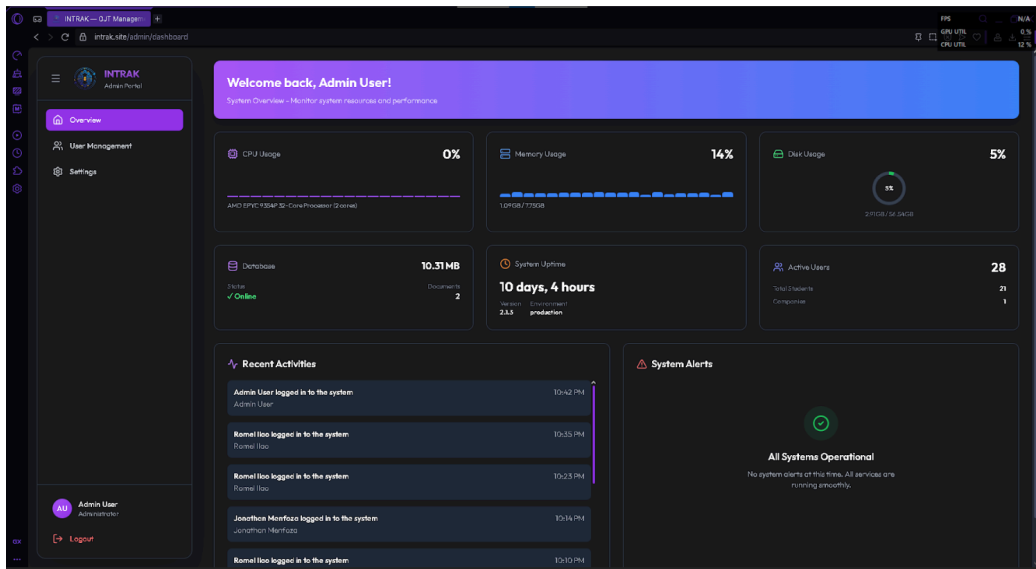
INSTRUCTOR DASHBOARD



INDUSTRY PARTNER



COORDINATOR



CHAPTER 3: DESIGN TRADE-OFFS

This chapter presents the evaluation of the design trade-offs based on the identified constraints. It discusses economic, safety, risk, environmental, and sustainability considerations for the three design alternatives. The chapter also includes a summary of normalized values and designers' rankings to support comparison. Lastly, it explains how these trade-offs influence the final design selection and presents a sensitivity analysis to validate the results.

Design trade-off refers to a situation where improving one design characteristic may result in the reduction of another. The researchers analyzed the tradeoff of the study using the constraints in the previous chapters of this study.

3.1 Summary of Constraints

Table of Constraints:

DESIGN 1	DESIGN 2	DESIGN 3
● Accessibility and WCAG Compliance ● Feedback and Notification System Enhancement ● Multi-Step Workflow Guidance ● Cross-Device Testing Documentation ● Brand Kit Documentation	● Technical Complexity ● Hardware Limitations ● Network Dependency ● Cost and Licensing Scalability ● Data Privacy and Compliance: ● System Reliability and Maintenance ● User Experience ● Security Limitations	● Hardware and Storage Constraints ● Cost Constraints ● Software and Compatibility Constraints ● Security and Compliance Constraints ● Operational Constraints

3.2 Trade Offs

The researchers conducted trade-off analysis to systematically compare the three proposed designs by evaluating their advantages and limitations under the given project constraints. This analysis is essential in determining the most practical and efficient solution that best aligns with the system goals and available resources.

3.2.1 Tradeoff 1: Economic

Design 1: Cloud-based with RPI NAS implementation Annual Cost

PROJECT INPUTS	Design 1	Cost
Hardware Implementation	Raspberry Pi NAS based	PHP 5,000.00
Domain name	site TLD	PHP 300.00
VPN service	Tailscale VPS(free tier)	PHP 0.00
Cloud Hosting Service	Hostinger VPS	PHP 12,228.00
TOTAL		PHP 17,528.00

The total estimated cost for implementing Design 1 of the INTRAK system is PHP 17,528.00. This includes the hardware implementation of the Raspberry Pi NAS-based setup at PHP 5,000.00, which provides the core infrastructure for storing internship records and integrating with cloud services. The domain name registration costs PHP 300.00, ensuring a professional and accessible web address for students, instructors, and administrators. The VPN service uses Tailscale's free tier, contributing no additional cost, while the cloud hosting service through Hostinger VPS accounts for PHP 12,228.00, supporting reliable and scalable access to the system.

Most of the cost is attributed to cloud hosting, followed by hardware, while domain registration and the VPN service are minimal contributors. This cost distribution

demonstrates that Design 1 provides a cost-effective balance between local hardware investment and cloud-based operations

Design 2: Cloud-Based Angular–Spring Boot Web Platform for INTRAK
Annual Cost

PROJECT INPUTS	Design 2	Cost
Hardware Implementation	No hardware implementation	N/A
Domain name	.com	PHP 1,200.00
VPN service	expressVPN	PHP 5,760.00
Cloud Hosting Service	Hostgator	PHP 10,080.00
TOTAL		PHP 18,040.00

The total estimated cost for Design 2 of the INTRAK system is PHP 18,040.00. This design does not require any physical hardware, as all components are hosted in the cloud. The domain name registration for a .com site costs PHP 1,200.00, providing a professional and accessible web address for users. The VPN service uses ExpressVPN at PHP 5,760.00, ensuring secure remote access for administrators and staff. The cloud hosting service is provided by Hostgator at PHP 10,080.00, which supports the web application, database, and storage entirely in the cloud.

The distribution of costs shows that the majority of expenses come from cloud-based services, particularly hosting and VPN subscriptions. The absence of hardware costs simplifies setup and reduces upfront investment, while ongoing subscriptions ensure the system remains secure, available, and reliable for all users.

Design 3: Cloud-based Architecture with dedicated NAS Storage
Annual Cost

PROJECT INPUTS	Design 3	Cost
Hardware Implementation	Commercial NAS device Synology DS223 2 bay DiskStation® NAS	PHP 25,467.00
Domain name	Site TLD	HP 300.00
VPN service	protonVPN	PHP 2,160.00
Cloud Hosting Service	cloudways	PHP 7,920.00

TOTAL	PHP 35,847.00
--------------	----------------------

The total estimated cost for Design 3 of the INTRAK system is PHP 35,847.00. This design includes a commercial NAS device, specifically the Synology DS223 2-bay DiskStation® NAS, costing PHP 25,467.00, which provides secure, redundant, and centralized storage for internship documents. The domain name registration for a site TLD costs PHP 300.00, ensuring an accessible web address for students, instructors, and administrators. The VPN service uses ProtonVPN at PHP 2,160.00 to provide secure remote access to both cloud resources and the NAS. The cloud hosting service is provided by Cloudways at PHP 7,920.00, which hosts the web application and database, supporting reliable online access for all users.

Cloud hosting and VPN services contribute additional operational costs, while the domain name represents a minimal investment. This setup ensures that the INTRAK system maintains secure, centralized, and reliable access to both structured and unstructured internship data

Proposed Design	Total Annual Cost
Design 1	PHP 17,528.00
Design 2	PHP 18,040.00
Design 3	PHP 35,847.00

The estimated total annual costs for the proposed INTRAK system designs vary depending on the selected architecture. Design 1 has a total cost of PHP 17,528.00, covering the Raspberry Pi NAS-based setup, domain registration, VPN service, and Hostinger VPS hosting. Design 2, a fully cloud-based architecture without NAS integration, is slightly higher at PHP 18,040.00, reflecting the costs of a .com domain, ExpressVPN subscription, and Hostgator cloud hosting. Design 3, which incorporates a commercial NAS device (Synology DS223), a site TLD, ProtonVPN, and Cloudways hosting, has the highest total at PHP 35,847.00, mainly due to the investment in dedicated NAS hardware.

3.2.2 Tradeoff 2: Safety

3.2.3 Tradeoff 3: Cybersecurity Risk Assessment

The researchers applied Common Vulnerability Scoring System (CVSS) to provide a clear and standardized way in identifying vulnerabilities, assess possible threats, and measure risk levels. This approach was applied to the INTRAK System to ensure that the platform is secure, reliable, and capable of handling sensitive internship data.

Design 1: Cloud-Based with NAS Integration Using a Raspberry Pi

Design 1 presents moderate to high cybersecurity risk due to its reliance on a Raspberry Pi as a NAS device. From a CVSS perspective, vulnerabilities related to hardware limitations, patch management, and physical security increase the attack surface. Raspberry Pi devices are not enterprise-grade and may lack advanced security features such as hardware encryption acceleration, intrusion detection, and robust access control mechanisms.

Additionally, the NAS functionality hosted on a Raspberry Pi introduces risks related to availability and integrity, especially under heavy file access or concurrent users. If the device is misconfigured or not regularly updated, vulnerabilities may be exploitable remotely, increasing the CVSS exploitability score. While suitable for low-cost deployment, this design requires strict administrative controls and continuous monitoring to mitigate cybersecurity risks.

Second Design : Cloud-Based Angular–Spring Boot Web Platform for INTRAK

Design 2 demonstrates a lower cybersecurity risk compared to Design 1, primarily due to its fully cloud-based architecture and absence of on-premise or locally managed storage hardware. Using cloud hosting reduces risks related to physical access, hardware failure, and local misconfiguration. From a CVSS standpoint, this design benefits from improved availability and confidentiality, as cloud providers typically implement strong baseline security controls.

Third Design : Cloud-Based Architecture with Dedicated NAS Storage Using Python Full Stack

Design 3 offers the lowest cybersecurity risk among the three designs when evaluated using CVSS. The use of a dedicated commercial NAS device provides enhanced security features such as RAID redundancy, access control, encryption support, and centralized permission management. These features reduce the likelihood and impact of data loss, unauthorized access, and system downtime.

The researchers conclude that, from a CVSS perspective, Design 3 presents the lowest overall cybersecurity risk among the proposed solutions. Its controlled access to dedicated NAS storage, well-structured backend logic, and clear separation of system layers significantly reduce exploitability. Although risks related to misconfiguration and network exposure remain, their potential impact is minimized through enterprise-grade storage controls, access permissions, and redundancy mechanisms. Overall, this design achieves a balanced combination of security, reliability, and scalability, making it the most suitable option for managing sensitive internship records in an institutional environment.

3.2.4 Tradeoff 4: Environmental Tradeoff

The researchers evaluated the environmental impact of the three proposed designs of the INTRAK System, focusing on energy use, resource efficiency, and sustainability. Key aspects such as hardware, operational costs, storage management, and maintenance were analyzed to show how the system reduces energy consumption, limits electronic waste, and supports eco-friendly academic technology practices.

Design 1: Cloud-based with NAS Integration using a Raspberry Pi

ASPECT	DESCRIPTION	BENEFIT/COST	IMPACT ON ENVIRONMENT
Hardware	Raspberry Pi NAS	Lower Energy consumption / Compact design	Reduces electricity and carbon footprint
Operational Cost	Power usage for system operation	Reduced electricity bills	Less environmental impact from energy production
Storage and Document Management	Centralized NAS storage replaces paper records	Decrease need for printing and paper storage	Conserve natural resources and reduces paper waste
Maintenance and Upgrades	Lightweight hardware with modular components	Easier to maintain and replace	Minimize electronic waste compared to large servers
Sustainability	Long-term deployment with low power devices	Continues to provide secure storage with minimal energy	Support eco-friendly practices and sustainability goals

1. Hardware

The choice of a Raspberry Pi NAS as the primary hardware for the INTRAK System offers significant environmental advantages compared to conventional servers. The

Raspberry Pi consumes significantly less electricity due to its low-power design, which directly reduces the system's overall energy consumption. This not only lowers operational costs but also decreases the carbon footprint associated with powering the system, making it a more sustainable solution for the institution.

2. Operational Cost

Lower energy consumption directly translates into cost savings for the institution. Operating a Raspberry Pi NAS requires far less electricity than running standard desktop or server hardware. This reduction in energy usage not only benefits the institution financially but also indirectly contributes to environmental conservation by minimizing energy demand and the associated emissions from electricity generation.

3. Storage & Document Management

The centralized NAS storage system in INTRAK reduces the need for physical documentation. By storing internship records and student documents digitally, the system significantly decreases paper consumption. This helps conserve natural resources and reduces the waste generated by traditional paper-based systems, making the overall process more environmentally friendly.

4. Maintenance & Upgrades

The modular and lightweight design of the Raspberry Pi makes it easy to maintain and upgrade individual components without replacing the entire system. This reduces electronic waste, which is a common issue with traditional servers that require complete replacement when components fail or become outdated. Efficient maintenance practices extend the system's lifecycle while minimizing environmental impact.

5. Sustainability

The INTRAK System's design proposed by the researchers supports long-term environmental sustainability. The combination of low power consumption, digital document management, and modular hardware ensures that the system remains energy-efficient and eco-friendly over time. By adopting such technologies, the institution can achieve secure and reliable data management while also promoting greener IT practices and reducing its ecological footprint.

Design 2: Cloud-Based Angular–Spring Boot Web Platform for INTRAK

ASPECT	DESCRIPTION	BENEFIT/COST	IMPACT ON ENVIRONMENT
Hardware	N/A	No dedicated hardware needed.	Less e-waste, lower energy use.
Operational Cost	Cloud hosting, domain, VPN fees.	Predictable, scalable costs.	Moderate energy use, often from efficient data centers.
Storage and Document Management	Cloud storage with backup.	Reliable access, minimal local storage.	Reduces paper use; energy for data centers.
Maintenance and Upgrades	Software updates and monitoring handled in cloud.	Minimal IT effort.	Less on-site energy and resource use.
Sustainability	Uses virtualized, efficient cloud infrastructure.	Scalable, reliable system	Low local resource use; supports e-waste reduction.

Hardware: The system relies entirely on cloud servers, with no dedicated on-site hardware required. This eliminates the need for purchasing and maintaining physical servers, which reduces upfront costs. Users access the system through their own devices, such as desktops, laptops, or mobile devices. This setup also minimizes electronic waste and reduces the institution’s energy consumption.

Operational Cost:

The main recurring expenses include cloud hosting, domain registration, and VPN services. These costs are predictable and scale according to the number of users and storage requirements. While cloud servers consume energy, most providers operate energy-efficient and often green-certified data centers. This makes operational costs manageable while minimizing environmental impact.

Storage and Document Management:

All internship documents and reports are stored in cloud storage with backup and redundancy mechanisms. This ensures that files are always accessible and reduces the need for local storage infrastructure. Paper use is minimized since documents are fully digital, contributing to environmental conservation. The cloud storage also allows easy scaling as data volume grows without significant additional hardware.

Maintenance and Upgrades:

Software updates, security patches, and system monitoring are handled by the cloud provider. This reduces the workload for institutional IT staff and eliminates the need for on-site maintenance. Centralized updates also ensure that the system remains secure and reliable. By avoiding frequent on-site maintenance, energy and resource consumption are further reduced.

Sustainability:

The system leverages virtualized cloud infrastructure that is scalable and efficient. It reduces the need for physical resources and limits electronic waste compared to on-premise setups. Energy-efficient cloud operations help minimize the environmental footprint of system usage. Overall, this approach supports long-term sustainability while maintaining high reliability and performance.

Design 3: Cloud-based Architecture with dedicated NAS Storage using Python full stack

ASPECT	DESCRIPTION	BENEFIT/COST	IMPACT ON ENVIRONMENT
Hardware	Synology DS223 2-bay NAS along with cloud server	Secure, centralized storage; higher upfront cost	NAS uses electricity; reduces dependence on larger servers
Operational Cost	NAS, drives, cloud hosting, domain, VPN	Predictable recurring costs	Moderate energy use; some e-waste from NAS
Storage and Document Management	NAS stores documents with cloud backup	Reliable access, secure backups	Reduces paper use; NAS consumes moderate energy
Maintenance and Upgrades	Firmware updates, drive health checks	Prevents data loss; prolongs NAS life	Low resource use; reduces hardware replacement
Sustainability	Combines NAS with cloud backups	Scalable, durable, reliable	Lowers reliance on large data centers; limits e-waste

Hardware:

The system uses a Synology NAS to centralize storage, providing secure and organized data management. While the NAS requires an upfront investment and consumes electricity, it reduces reliance on large server infrastructures. Users can access files efficiently, ensuring smooth operations. Overall, the hardware setup balances cost, functionality, and reliability.

Operational Cost:

Costs include the NAS, storage drives, cloud hosting, domain, and VPN services. These are predictable and allow budgeting for both one-time and recurring expenses. Although the NAS adds some energy consumption, the system avoids expensive on-premise server setups. The overall cost ensures a reliable, manageable infrastructure.

Storage & Document Management:

All internship documents are stored on the NAS with optional cloud backup for redundancy. This ensures safe and consistent access while minimizing data loss risks. Digital storage reduces the need for paper, making document management eco-friendly. The NAS also allows easy scaling as document volume grows.

Maintenance & Upgrades:

Routine firmware updates and drive checks help maintain NAS health and prolong its lifespan. Preventive maintenance reduces unexpected failures and ensures data integrity. While some IT oversight is needed, resource usage is minimal compared to replacing hardware frequently. This keeps the system reliable and cost-effective.

Sustainability:

The combination of NAS and cloud storage ensures long-term reliability and scalability. Energy-efficient cloud usage complements the NAS, reducing environmental impact compared to fully on-premise solutions. This setup limits e-waste while providing robust, sustainable operations for institutional use.

Environmental Analysis

The researchers tabulate the results of the proposed designs of the system. It shows that all approaches reduce paper usage and promote digital document management, thereby limiting waste. Design 1 (Raspberry Pi NAS-based) consumes very little power due to the low-energy Raspberry Pi, and its modular hardware allows upgrades without generating significant e-waste. Design 2 (Cloud-based Angular–Spring Boot) relies entirely on cloud infrastructure, minimizing on-site hardware and maintenance, though energy use depends on the data centers. Design 3 (Dedicated NAS with Python full stack) centralizes storage using a Synology NAS, which moderately increases local

energy use but provides long-term reliability, redundancy, and reduced paper consumption.

Among the three, Design 1 offers the most environmentally friendly solution due to its extremely low power consumption, minimal hardware footprint, and sustainable lifecycle management. Design 2 follows closely with low local resource use, while Design 3, although efficient and reliable, has higher energy demands because of the dedicated NAS device.

3.2.5 Tradeoff 5: Sustainability: Power Consumption, System Lifespan, and Maintainability

This researcher considers the sustainability trade-offs of the INTRAK System by examining power consumption, system lifespan, and maintainability of the three designs. These factors together highlight the system's environmental and operational sustainability.

Design 1: Cloud-Based with NAS Integration using a Raspberry Pi

Power Consumption: Design 1 has low power requirements due to the lightweight nature of the Raspberry Pi and USB-based storage. The system consumes significantly less electricity than traditional desktop servers or enterprise NAS devices, making it energy-efficient for continuous academic use. Low power consumption not only reduces operational costs but also lowers the environmental footprint, aligning with sustainability goals. Additionally, the reduced heat output from the Raspberry Pi helps maintain device reliability and eliminates the need for extensive cooling systems.

System Lifespan: The modularity of the Raspberry Pi and external storage components allows for easy upgrades or replacements without replacing the entire system. Individual parts such as SD cards or USB drives can be swapped before reaching the end of their expected life, extending the overall operational lifespan to

approximately 5–7 years. This design ensures long-term reliability while minimizing electronic waste, supporting sustainable IT practices within an institution.

Maintainability: Maintenance in this design is relatively simple. Software updates, system monitoring, and backup management can be performed with minimal technical effort. The lightweight setup also reduces the likelihood of system downtime, ensuring uninterrupted access to internship records. Routine checks of storage health and timely replacement of drives help maintain optimal system performance over time.

Design 2: Cloud-Based Angular–Spring Boot Web Platform for INTRAK

Power Consumption: In Design 2, local power usage is minimal because the system primarily relies on cloud infrastructure. The energy footprint depends largely on the efficiency of the cloud data centers, which typically employ advanced power management, virtualization, and optimized cooling solutions. Users' devices only handle lightweight tasks, further reducing local electricity consumption. This design is highly suitable for institutions that aim to lower energy use while providing scalable access to multiple users.

System Lifespan:

Because there is no dependency on on-site hardware, the system can operate continuously without the limitations of physical equipment aging. Cloud servers can be scaled, upgraded, or replaced by the service provider without any impact on the institution. This allows the system to remain up-to-date with the latest technologies and security measures, effectively giving it an indefinite lifespan as long as cloud services are maintained.

Maintainability:

Maintainability is one of the strongest advantages of this design. Cloud service providers handle software updates, backups, and server monitoring, reducing the need for dedicated IT staff. Any issues can often be resolved remotely without disrupting users. Centralized management simplifies administrative work, ensures consistent performance, and allows automated scaling to meet varying user demands.

Design 3: Cloud-Based Architecture with Dedicated NAS Storage using Python Full Stack

Power Consumption: Design 3 uses a dedicated Synology NAS, which requires moderate local power for continuous operation. While higher than a Raspberry Pi or fully cloud-based system, this power consumption is justified by the reliable storage, redundancy, and high availability it provides. The combination of local NAS and optional cloud backup balances performance with energy use, ensuring critical data is always accessible while maintaining moderate operational costs.

System Lifespan: The Synology NAS is designed for long-term use, with features such as RAID configurations, regular firmware updates, and health monitoring of drives. With proper care, the system can remain operational for many years, and failing components can be replaced individually without replacing the entire NAS. This extends the lifespan of the system significantly compared to traditional all-in-one hardware setups and ensures data remains protected over the long term.

Maintainability: This design requires more active maintenance than a fully cloud-based solution. IT personnel must monitor NAS health, perform periodic backups, update software, and replace drives as necessary. However, the modularity of the NAS allows maintenance tasks to be performed without major disruptions to the system. While more resource-intensive, this approach ensures data integrity, resilience against hardware failure, and reliable access for all users. Maintainability in an academic setting.

3.3 Influence of the Design Tradeoffs on the Final Design

Economic Trade-off: The researchers prioritized cost-effectiveness by selecting a Raspberry Pi NAS-based storage solution instead of a traditional server. This choice reduced initial investment and operational expenses while still providing sufficient performance for managing student internship data.

Safety Trade-off: Security measures, such as user authentication and access controls, were implemented to protect sensitive information. The design ensured that these safety features did not complicate system use, maintaining a balance between protection and user convenience.

Risk Trade-off: The system focused on mitigating high-impact vulnerabilities, such as unsecured NAS directories or outdated software packages, while lower-risk issues received fewer resources. This allowed efficient allocation of security measures without overcomplicating the system.

Environmental Trade-off: The INTRAK System was designed with low-power hardware and digital document management to minimize energy consumption and reduce paper usage. These measures limit the system's carbon footprint and contribute to sustainable IT practices.

Sustainability Trade-off: Long-term maintainability and system lifespan were emphasized through modular software design, well-documented code, and routine maintenance. These practices ensure the system remains functional, adaptable, and reliable over time, supporting institutional needs without frequent overhauls.

By carefully considering these trade-offs, the researchers developed a final INTRAK design that is cost-efficient, secure, environmentally responsible, and sustainable, providing a reliable platform for managing internship data in an academic setting.

Summary of Normalized Values and Design Selection

The trade-off scores are summarized in the following table:

Design Criteria	Weight	Design 1 Cloud + RPI NAS	Design 2 Full Cloud	Design 3 Cloud + Commercial NAS
Economic Cost	20%	8	7	10
Safety/Security	25%	9	6	8
Risk/Reliability	20%	9	8	6
Environmental	15%	7	9	8

Sustainability	20%	9	8	6
Weighted Score	100%	8.50	7.45	7.60

Weighting Rationale:

Safety/Security (25%): Given the highest weight because student records must be kept private and protected, in line with the Data Privacy Act.

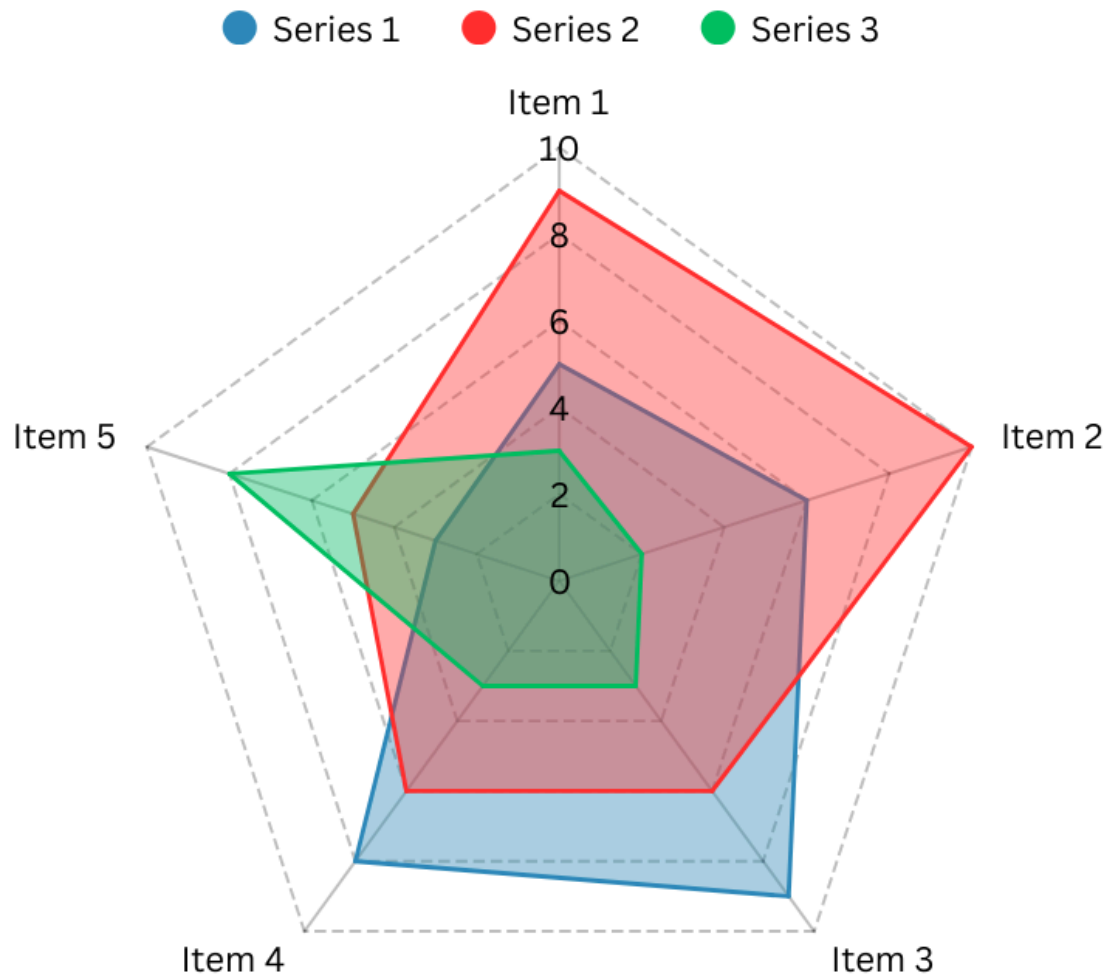
Economic (20%): Important due to budget limits, but cost is not the main factor in system design.

Risk/Reliability (20%): High weight because the system needs to be available and reliable for daily academic use.

Sustainability (20%): Important to ensure the system can operate and be maintained over many years.

Environmental (15%): Considered important, but less critical than security, reliability, and system performance.

RADAR CHART



Design 1: Cloud-based with NAS Integration using a Raspberry Pi

Design 1 demonstrates strong and consistent performance across the most critical criteria. It scores highest in Safety/Security (9), Risk/Reliability (9), and Sustainability (9), reflecting its suitability for protecting sensitive student data and supporting long-term operations. While its scores for Economic Cost (8) and Environmental impact (7) are slightly lower, the design remains well-balanced overall. On the radar chart, this creates a large, evenly spread polygon, visually showing its robustness and alignment

with institutional priorities. The weighted score of 8.50 confirms that Design 1 is the top choice for the project.

Design 2: Cloud-Based Angular–Spring Boot Web Platform for INTRAK

Design 2 performs well in Environmental (9) and Economic Cost (7), indicating lower hardware requirements and simplified operational management. However, it scores lower in Safety/Security (6) and slightly lower in Sustainability (8), reflecting the risks of relying on external cloud providers and limited local control of sensitive data. On the radar chart, this creates an irregular shape with noticeable dips in security-related areas, signaling vulnerabilities. Its weighted score of 7.45 shows that, while cost-effective and environmentally friendly, Design 2 is less suitable for institutional priorities where data privacy and reliability are critical.

Design 3: Cloud-based Architecture with dedicated NAS Storage using Python full stack

Design 3 shows high scores in Economic Cost (10) and Environmental impact (8), highlighting efficiency and lower operational impact. However, it scores lower in Risk/Reliability (6) and Sustainability (6), suggesting potential weaknesses in long-term system stability and reliability. The radar chart shows a narrower, uneven polygon, with peaks in cost and environment but significant gaps in critical operational areas. With a weighted score of 7.60, Design 3 performs slightly better than Design 2 overall, but it is less balanced and less robust than Design 1 in addressing key institutional needs.

Conclusion

The radar chart shows that Design 1 (Cloud + RPI NAS) is the most balanced and robust option, scoring highest in security, reliability, and sustainability, making it well-suited for institutional priorities. Design 2 (Full Cloud) is strong in environmental impact and cost but weaker in security and long-term control, making it less suitable. Design 3 (Cloud + Commercial NAS) performs well in cost and environmental criteria but has

lower reliability and sustainability scores. The chart confirms that Design 1 is the optimal choice for the project.

Design Selection Decision

Design 1 (Hybrid Cloud with NAS Storage) is chosen as the best solution, with a weighted score of 8.50 out of 10, performing better than the other designs.

This decision is based on the following reasons:

1. **Meets Project Constraints:** Design 1 keeps student data under institutional control while still allowing secure remote access when needed.
2. **Strong Security:** The design provides high data protection, which is essential for safeguarding student records and complying with data privacy requirements.
3. **Balanced Performance:** Design 1 performs well across all evaluation criteria and does not show major weaknesses. It avoids the limited scalability of Design 3 and the dependence on external providers seen in Design 2.
4. **Good Institutional Fit:** The design matches common university IT practices and works well with existing network and backup systems.
5. **Supports Future Growth:** The NAS-based setup allows easy expansion as more users, departments, and data are added in the future.

Although Design 3 scores highest in terms of cost efficiency, it performs poorly in scalability-related areas such as risk and sustainability. This makes it unsuitable for a system expected to grow over time.

Design 2, while offering advantages like automatic scaling and reduced maintenance, does not meet the requirement for local data storage and creates reliance on external service providers, which is not acceptable for an academic institution.

Influence of the Design Trade-Off on the Final Design

The trade-off analysis had a direct impact on how the final INTRAK system was designed and implemented.

Storage Architecture Decision

The choice to use NAS storage strongly influenced the overall system structure. Uploaded files are stored on the NAS using the SMB protocol, while the web system uses secure HTTPS connections for remote access. The system includes the needed network permissions and file access tools to communicate with the NAS. Error handling is designed to manage storage-related issues such as network interruptions or lack of available storage space.

API Design

The storage approach affected how the system's API was designed. During file uploads, the backend first saves the file to the NAS and then stores the file details in the PostgreSQL database. This two-step process ensures that both the file and its information are properly recorded. Transaction controls are used to keep the file storage and database records consistent.

Security Implementation

Security considerations played a major role in the final design. All file actions require proper user authorization before they are allowed. The system records file access activities, including the user, time, and type of action performed. Access to the NAS is limited to system service accounts, preventing direct access by students or faculty. Secure connections, firewalls, and VPN requirements are enforced for all remote access.

Capacity Planning

Sustainability considerations guided storage capacity planning. The NAS was configured with an initial usable capacity of 12TB to support projected system growth over five years. This estimate is based on expected document sizes and user numbers, with additional space included to allow safe long-term operation.

Infrastructure Requirements

The trade-off analysis also influenced hardware selection. The final design uses enterprise-grade NAS hardware with RAID 5 for improved reliability. Consumer-grade devices were excluded because they do not meet the system's reliability and risk requirements.

Backup Strategy

Using NAS storage allows easy integration with existing institutional backup systems. The backup plan includes daily incremental backups and weekly full backups of stored documents. This approach supports long-term data protection and recovery needs.

Deployment Architecture

The NAS is placed on the same network as the application server to improve file transfer speed and reduce delays. Gigabit Ethernet connectivity ensures smooth system performance. A secure internet gateway is used to allow controlled remote access for authorized external users.

Rejected Alternative Features

Pure cloud-based storage was not selected, and features such as public content delivery networks were excluded. Instead, remote access needs are met through the hybrid NAS setup, allowing secure access while keeping institutional data under local control.

3.4 Sensitivity Analysis

Sensitivity analysis evaluates the robustness of the design selection decision under varying weightings of the evaluation criteria. By systematically adjusting criterion weights, the analysis determines whether Design 1 remains optimal across different priority scenarios or if the decision is fragile to weight changes.

Ten sensitivity iterations explore different weighting schemes, representing alternative stakeholder priority rankings. Each iteration maintains total weight of 100% while varying individual criterion emphasis.

Iteration	Weighting	Design 1	Design 2	Design 3
1	Economic-Focused (10-9-8-7-6)	8.60	7.05	8.50
2	Security-Focused (6-10-7-8-9)	9.05	7.60	7.25
3	Environmental-Focused (6-8-10-7-9)	8.45	8.20	6.95
4	Reliability-Focused (9-7-10-8-6)	8.80	7.75	7.35
5	Sustainability-Focused (8-7-6-9-10)	8.70	7.90	7.05
6	Balanced Uniform (10-9-6-7-8)	8.60	7.60	7.60
7	Cost-Minimization (7-9-8-6-10)	8.35	7.00	8.85
8	Risk-Averse (8-6-9-10-7)	8.75	7.65	7.30
9	Cloud-Friendly (9-10-8-6-7)	8.10	7.90	7.50
10	Operations-Focused (7-10-9-6-8)	8.90	7.55	7.10

Design 1: Cloud-based with NAS Integration using a Raspberry Pi

Design 1 consistently performs well across most weighting scenarios. It scores highest in security-focused iterations (9.05 in Iteration 2) and remains strong in reliability, sustainability, and balanced scenarios. Even in cost-focused iterations, its score is competitive (8.60 in Iteration 1), showing that it maintains good performance across different priorities. The radar chart would show a large, fairly even polygon, indicating

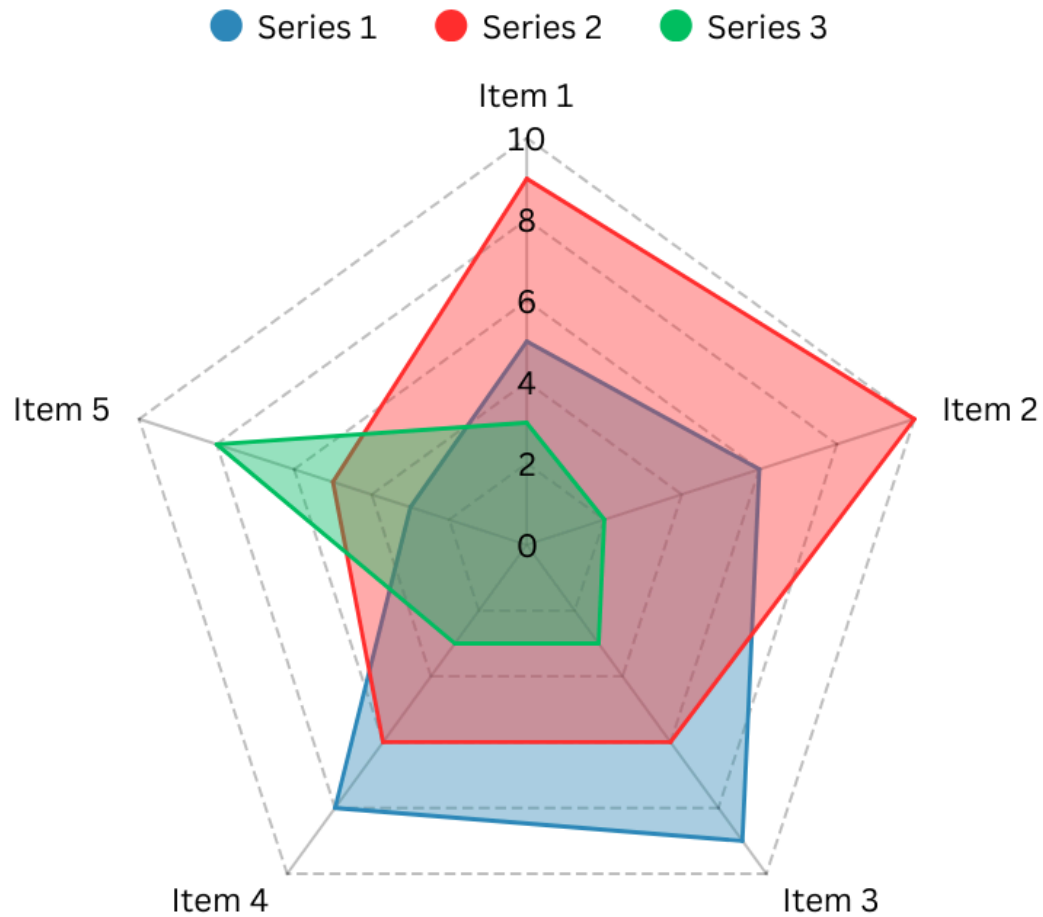
a robust and stable design that aligns with institutional priorities and regulatory requirements.

Design 2: Cloud-Based Angular–Spring Boot Web Platform for INTRAK

Design 2 performs moderately across all iterations. It scores relatively higher in environmental-focused (8.20 in Iteration 3) and balanced iterations, but it consistently underperforms in economic-focused (7.05 in Iteration 1) and security-focused scenarios (7.60 in Iteration 2). On the radar chart, the polygon would appear smaller and irregular, reflecting its vulnerability in critical areas like security and risk management, making it less suitable for an institution where data protection is a priority.

Design 3: Cloud-based Architecture with dedicated NAS Storage using Python full stack

Design 3 shows strong performance in cost-sensitive iterations (8.85 in Iteration 7) but lower scores in security, sustainability, and operations-focused scenarios (as low as 7.05 in Iteration 5 and 7.10 in Iteration 10). The radar chart would reveal a spiky, uneven shape, highlighting high variation depending on criteria weighting. This indicates that while Design 3 may be attractive for cost efficiency, it is less balanced and less robust compared to Design 1.



Conclusion

The radar chart confirms that Design 1 is the most balanced and reliable design, maintaining high scores across most priority scenarios. Its polygon is broad and even, representing consistent performance and robustness to changes in weighting. Design 2 is weaker and irregular, particularly in security and risk, while Design 3 is variable, performing well in cost-focused scenarios but poorly in reliability and sustainability. The radar chart analysis supports the selection of Design 1 (Cloud + RPI NAS) as the optimal solution for institutional needs and project requirements.

Minimum Margin of Victory:

Design 1's smallest advantage appears in Iteration 1, where the focus is mainly on cost (8.60 vs. 8.50), a difference of only 0.10 points. However, this case places too much

importance on cost and reduces the importance of security and sustainability, which does not reflect the institution's actual priorities.

Maximum Margin of Victory:

Design 1's largest advantage occurs in Iteration 2, which focuses on security (9.05 vs. 7.60), resulting in a difference of 1.45 points. This scenario closely matches the institution's needs, where protecting data is the top concern.

Stability Analysis:

Design 1 shows more consistent results across all test cases. Its score changes only slightly compared to Design 2 and Design 3. Design 3 shows large changes in score, meaning its performance depends too much on cost-related weighting.

Constraint Violation Impact

Design 2 does not meet the requirement for local data storage, which is a strict project rule. Because of this, it cannot be selected, even if it were to achieve a higher numerical score.